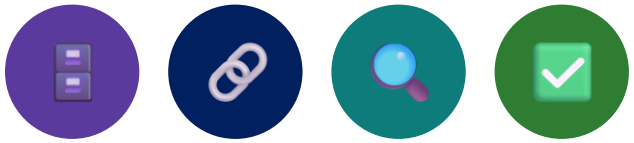
"Focus on your business logic -- JPA takes care of the persistence."

MODULE 39

Spring Data JPA and PostgreSQL Integration

Map Java entities to PostgreSQL tables with Spring Data JPA, and build repositories, queries, relationships, and transactions the right way.



TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 39 OVERVIEW

Build a real Spring Data JPA persistence layer on top of the PostgreSQL schema from Module 37.

Northstar CRM's Spring Boot backend needs entities, repositories, and transactions wired to the exact PostgreSQL schema designed in Module 37.

DURATION & LAB

1.5 Hours Theory

JPA architecture, entities, repositories, queries, relationships, transactions, and migrations.

2 Hours Lab

Build entities, repositories, and tests against a real PostgreSQL database.

Scenario

Northstar CRM's Spring Boot backend, persisting through Spring Data JPA to PostgreSQL.

MODULE ARC

Understand JPA

ORM, JPA, Hibernate, and Spring Data JPA -- how each layer fits together.

Map & Query

Entities, repositories, derived queries, JPQL, and native SQL.

Relate & Transact

Relationships, fetch strategies, transactions, migrations, and testing.

KEY TAKEAWAY Total time: 3.5 hours (1.5 hours theory + 2 hours hands-on lab). By the end, you will build a tested Spring Data JPA persistence layer against real PostgreSQL.

WHY ORM MATTERS

Object-Relational Mapping bridges object-oriented Java code and relational PostgreSQL tables -- domain objects instead of boilerplate SQL.

Without ORM	With ORM
Write repetitive SQL for every CRUD operation.	Work with entities and repositories.
Handle result sets and type conversions by hand.	Automatic mapping and persistence.
Tight coupling to the exact database schema.	Looser coupling; schema evolves more safely.
More code, more room for mistakes.	Less code, higher productivity.

KEY TAKEAWAY ORM doesn't replace SQL -- it abstracts it. You still benefit from everything Module 37 and 38 taught about PostgreSQL; ORM just removes the boilerplate around it.

Why ORM Matters

ORM (Object-Relational Mapping) bridges the gap between **object-oriented code** and **relational databases**.

THE PROBLEM ORM SOLVES

```
user.setName("Alice");
user.setEmail("alice@corp.com");
userRepository.save(user);
}
```

Object-Oriented World
Classes, Objects, Methods

ORM maps objects to database tables and vice versa

USERS

ID	NAME	EMAIL
1	Alice	alice@corp.com
2	Bob	bob@corp.com
...		

Relational World
Tables, Rows, Columns

KEY BENEFITS

Developer Productivity
Write less SQL. Focus on business logic, not boilerplate.

Type Safety
Compile-time checks reduce runtime errors and improve stability.

Maintainability
Changes in schema or code are easier to manage.

Portability
Switch databases with minimal changes to application code.

Advanced Features
Leverage caching, lazy loading, transactions, and more.

WHAT ORM PROVIDES

Abstraction from SQL
Avoid writing repetitive SQL for CRUD operations.

Object-Relational Mapping
Map classes to tables and relationships to joins.

Data Access Layer
Repositories provide a clean, consistent API for data access.

Transaction Management
Simplify ACID transactions with declarative support.

Performance Optimization
Use caching, batching, and fetching strategies.

ORM doesn't replace SQL — it empowers you to use the right level of abstraction for the job.

MODULE LEARNING OBJECTIVES

By the end of this module, you will build a tested Spring Data JPA persistence layer against real PostgreSQL.

1. Understand the Stack -- explain how JPA, Hibernate, and Spring Data JPA relate.
2. Configure the Connection -- wire Spring Boot's DataSource to PostgreSQL correctly.
3. Map Entities -- use @Entity, @Table, @Id, and @Column to map classes to tables.
4. Build Repositories -- use JpaRepository, derived queries, JPQL, and native SQL.
5. Model Relationships -- map every cardinality with the correct owning side and fetch type.
6. Manage Transactions -- apply @Transactional and understand rollback behavior.
7. Version the Schema -- explain how Flyway keeps migrations reproducible.
8. Test the Layer -- write repository tests that run against a real PostgreSQL database.

KEY TAKEAWAY Goal: build entities, repositories, and transactions that map cleanly onto the PostgreSQL schema from Module 37, verified by real tests.

MODULE 39

Learning Objectives

By the end of this module, you will be able to:

01 Explain the role of ORM, JPA, and Spring Data JPA in application development.

02 Configure Spring Boot to connect and integrate with PostgreSQL.

03 Create JPA entity classes and map them to database tables.

04 Create Spring Data JPA repositories and perform CRUD operations.

05 Use derived query methods, JPQL, and native PostgreSQL queries.

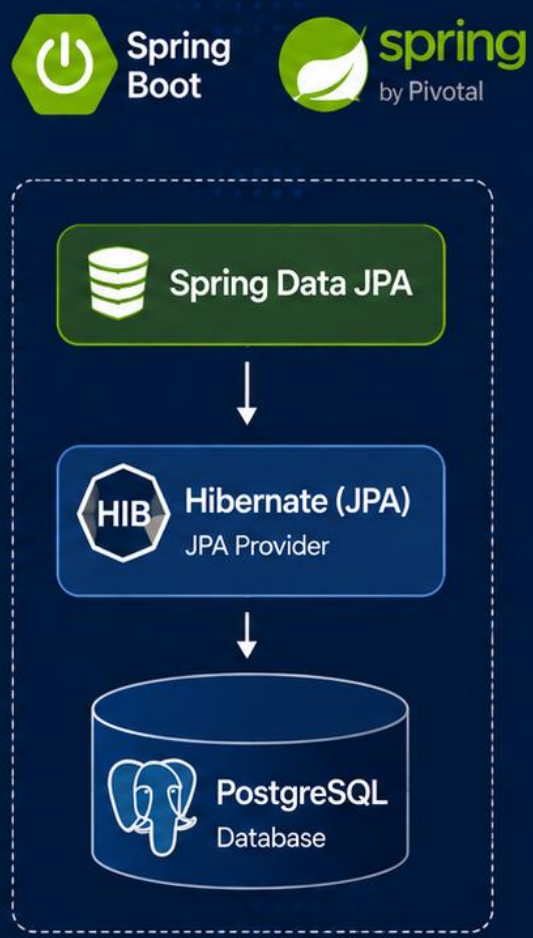
06 Model and implement relationships between entities.

07 Understand and apply fetch strategies to optimize performance.

08 Implement pagination, sorting, and transactional boundaries.

09 Handle exceptions and constraint violations from PostgreSQL.

10 Apply best practices for building robust, maintainable persistence layers.



PERSISTENCE ARCHITECTURE

A layered stack turns Java method calls into PostgreSQL SQL and back -- each layer has exactly one job.

1 Application

Controllers and services call repository methods -- no SQL, no JDBC in sight.

2 Spring Data JPA

Implements repository interfaces automatically at runtime.

3 JPA / Hibernate

The specification and its provider -- entity state tracking and SQL generation.

4 PostgreSQL

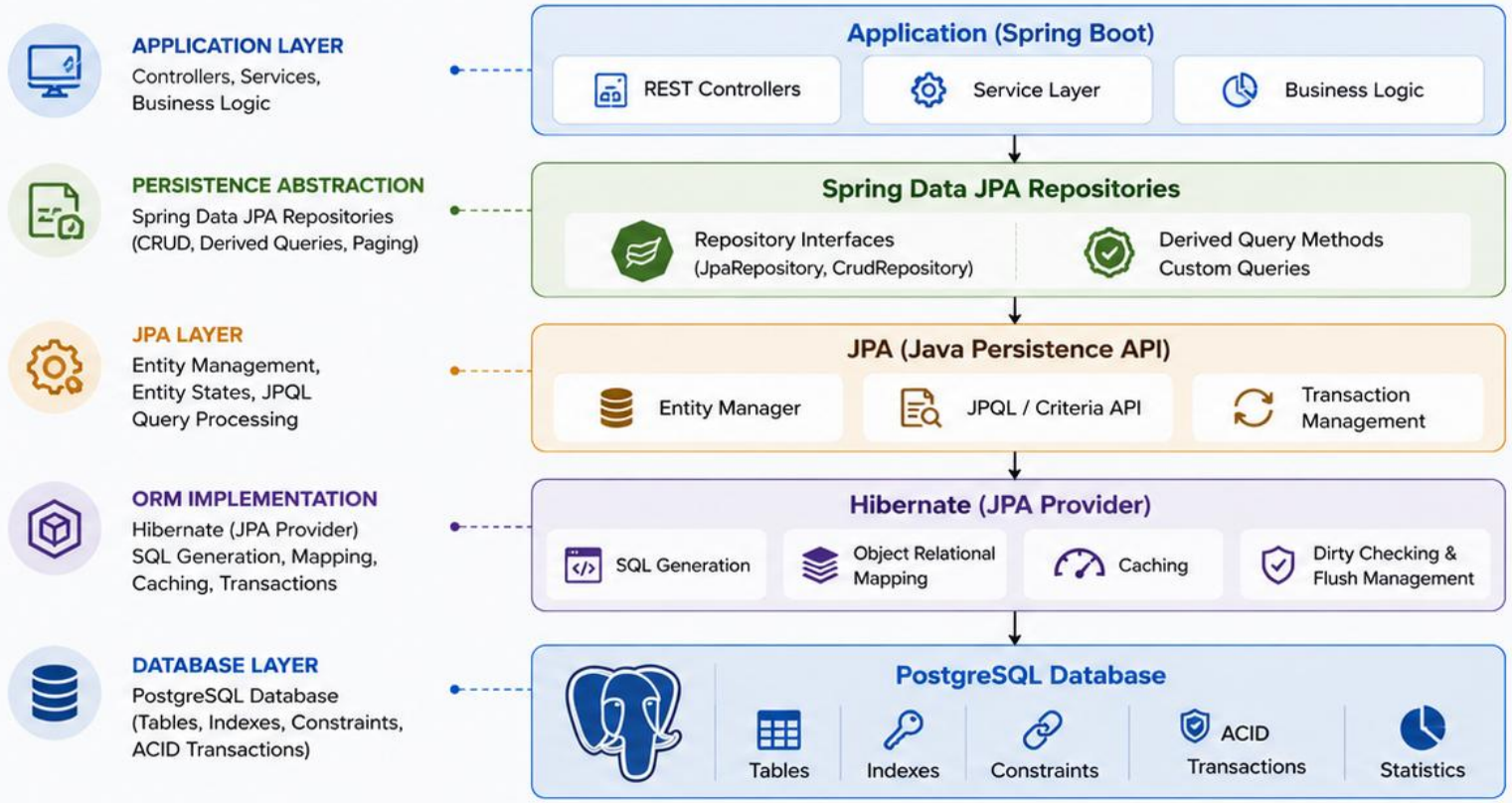
Executes the generated SQL and returns results.

KEY TAKEAWAY Each layer has exactly one job: application code stays SQL-free, Spring Data JPA generates repository implementations, JPA/Hibernate tracks entity state and produces SQL, PostgreSQL executes it.

MODULE 39

Persistence Architecture

Spring Data JPA provides an abstraction layer over JPA/Hibernate to simplify data access and integrate seamlessly with PostgreSQL.



HOW DATA FLOWS

1. Application calls Repository methods
2. Spring Data JPA creates query (derived or custom)
3. JPA (Entity Manager) processes the query and manages entities
4. Hibernate translates JPQL to SQL and executes it
5. Results are mapped to entities and returned to the application

KEY BENEFITS

- Reduces boilerplate data access code
- Consistent and type-safe queries
- Automatic entity mapping
- Transaction management integration
- Database independence (portable code)
- Better productivity and maintainability

Spring Data JPA sits between your application and the database, providing powerful abstractions while leveraging JPA and Hibernate to deliver high-performance persistence with PostgreSQL.

Focus on your business logic, not on writing SQL.

JPA OVERVIEW

JPA (Java Persistence API) is a Jakarta EE specification -- a standard, not an implementation -- defining how Java objects map to relational tables.

JPA is a specification -- a set of interfaces and rules -- not code that runs by itself.
It defines annotations (@Entity, @Id, @Column...), the EntityManager API, and JPQL.
Any JPA provider (Hibernate, EclipseLink) can implement the same specification.
This course uses Hibernate, the default and most widely used JPA provider, covered next.

KEY TAKEAWAY JPA is the standard; Hibernate, covered next, is the implementation this course runs -- understanding this distinction is the single most important architectural fact this module teaches.

MODULE 39

JPA Overview

JPA (Java Persistence API) is a specification that defines a standard way to map Java objects to relational database tables and manage persistence.

WHAT IS JPA?

- A Java specification (Jakarta Persistence) for object-relational mapping (ORM).
- Provides a set of APIs and annotations to persist, retrieve, update, and delete Java objects.
- Works with any JPA provider (e.g., Hibernate, EclipseLink).
- Simplifies database access and promotes portability across different persistence implementations.

JPA BENEFITS

- Reduces boilerplate JDBC code.
- Improves developer productivity.
- Database independence.
- Type-safe and object-oriented queries.
- Built-in caching and dirty checking.
- Transaction management integration.
- Easy maintenance and scalability.

HOW JPA WORKS

JPA handles SQL generation, entity mapping, transactions, and connection management behind the scenes.

JPA ANNOTATION EXAMPLE

```
@Entity
@Table(name = "employees")
public class Employee {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(name = "name", nullable = false, length = 100)
  private String name;

  @Column(name = "email", unique = true)
  private String email;
}
```

Employee.java

id : Long
name : String
email : String

employees (Table)

Column	Type
id	BIGINT
name	VARCHAR(100)
email	VARCHAR(100)

KEY JPA COMPONENTS

- **Entity** — Java class mapped to a database table using annotations.
- **Entity Manager** — Primary interface for managing entities.
- **Persistence Context** — A cache of managed entities.
- **JPQL (Java Persistence Query Language)** — Object-oriented query language.
- **Transactions** — Ensure data consistency and integrity.

JPA PROVIDERS (Examples)

HIBERNATE

eclipse LINK

Others: OpenJPA, Apache DataNucleus

JPA provides a powerful abstraction layer that bridges the gap between Java applications and relational databases using standard APIs and annotations.

Learn once, use anywhere — write persistence code that lasts.

HIBERNATE OVERVIEW

Hibernate is the JPA provider this course uses -- the concrete implementation that actually generates and executes SQL.

Implements JPA

Provides real, working code behind every JPA annotation and interface.

Tracks Entity State

Knows which entities are new, changed, or deleted since they were loaded.

Generates SQL

Produces the actual INSERT, UPDATE, SELECT, and DELETE statements PostgreSQL executes.

Adds Extras

Second-level caching, batching, and PostgreSQL-specific dialect handling beyond the bare JPA spec.

KEY TAKEAWAY Hibernate is what actually runs -- it implements JPA's interfaces, tracks every entity's state, and is the component ultimately responsible for the SQL Module 38's tools would diagnose.

MODULE 39

Hibernate Overview

HIBERNATE
Powered by JPA

Hibernate is the most popular JPA implementation. It provides a robust, performance-optimized **ORM framework** for mapping Java objects to relational databases.

WHAT IS HIBERNATE?

- A high-performance ORM framework that implements JPA specifications and provides additional features.
- Handles object-relational mapping, SQL generation, caching, transactions, and more.
- Designed for scalability, flexibility, and enterprise applications.

HIBERNATE BENEFITS

- High performance and scalability
- Automatic SQL generation
- Rich feature set (caching, lazy loading, batching)
- Database independence
- Strong transaction support
- Active open-source community
- Seamless integration with Spring Data JPA

HOW HIBERNATE WORKS

Java Application → JPA / Hibernate API → Hibernate Engine → Database

Hibernate translates object operations into SQL statements, executes them, and maps the results back to Java objects.

HIBERNATE ARCHITECTURE

Application Layer	Application (Entities, Services, DAOs, Repositories)			
Hibernate Layer	Configuration	Mapping Metadata	SessionFactory	Hibernate Services
JDBC Layer	JDBC / Connection Pool			
Database	Relational Database (PostgreSQL)			

KEY HIBERNATE FEATURES

- **Automatic SQL Generation** — Generates optimized SQL for all CRUD operations.
- **Lazy Loading** — Loads related data only when needed.
- **Caching** — First-level (Session) and Second-level caching for better performance.
- **Batch Processing** — Efficient handling of bulk insert/update/delete.
- **Transaction Management** — Integrates with JTA and resource-local transactions.
- **Query Support** — HQL, Criteria API, Native SQL, and more.

JPA vs HIBERNATE

JPA (Specification)	Hibernate (Implementation)
Defines standard APIs	Provides concrete implementation
Provider independent	Vendor specific optimizations
Limited feature set	Rich and advanced features
E.g., EntityManager API	Session, StatelessSession, etc.

Hibernate brings your Java objects to life in the database with powerful mapping, performance, and flexibility.

Hibernate + JPA + Spring Data JPA = Productive, maintainable, and high-performance persistence.

SPRING DATA JPA OVERVIEW

Spring Data JPA sits on top of JPA and Hibernate, eliminating repository boilerplate entirely.

Repository Interfaces

Extend JpaRepository -- Spring generates a working implementation automatically.

Derived Queries

Method names like findByStatus(...) are parsed into real queries -- no code needed.

Pagination & Sorting

Pageable and Sort are built in, no manual LIMIT/OFFSET tracking.

Spring Ecosystem Fit

Works seamlessly with @Transactional, Spring Security, and Spring Boot auto-configuration.

KEY TAKEAWAY Spring Data JPA is the productivity layer this course actually codes against -- students write an interface; Spring generates the implementation.

MODULE 39

Spring Data JPA Overview

spring by Pivotal

Spring Data JPA

Spring Data JPA is a part of the Spring Data project that **makes it easy to build data access layers** using JPA and Hibernate.

WHAT IS SPRING DATA JPA?

- An abstraction over JPA and Hibernate that simplifies repository development.
- Provides a set of interfaces and features to reduce boilerplate code.
- Boosts developer productivity with convention over configuration.
- Integrates seamlessly with Spring Boot and the Spring ecosystem.

HOW IT WORKS

Spring Data JPA automatically implements repository methods, creates queries, interacts with the database through JPA, and returns mapped entities.

KEY FEATURES

- **Repository Abstraction** — Define repository interfaces by extending JpaRepository.
- **Derived Query Methods** — Create queries automatically from method names.
- **Custom Queries** — Use @Query for JPQL or native SQL queries.
- **Pagination & Sorting** — Built-in support for Pageable and Sort.
- **Auditing Support** — Track created/modified dates and users.

KEY BENEFITS

- Reduces boilerplate DAO code
- Out-of-the-box CRUD operations
- Derived query methods from method names
- Pagination and sorting support
- Consistent exception translation
- Easy integration with transactions and security

REPOSITORY EXAMPLE

```
@Repository
public interface EmployeeRepository
       extends JpaRepository<Employee, Long> {

  // Derived query methods
  List<Employee> findByDepartment(String department);
  Optional<Employee> findByEmail(String email);
  List<Employee> findBySalaryGreaterThan(BigDecimal salary);

  // Custom query method
  @Query("select e from Employee e where e.status = :status")
  List<Employee> findByStatus(@Param("status") String status);
}
```

What you get automatically:

- Implementation of CRUD methods
- Query generation from method names
- Pagination & sorting
- Transactional support
- Exception translation
- Entity mapping and persistence

SPRING DATA JPA STACK

Spring Boot
↓
Spring Data JPA
↓
HIBERNATE
↓
PostgreSQL

Spring Data JPA abstracts the complexity of JPA and Hibernate, letting you focus on building business logic, not data access code.

Write less. Do more. That's the power of Spring Data JPA.

POSTGRESQL INTEGRATION ARCHITECTURE

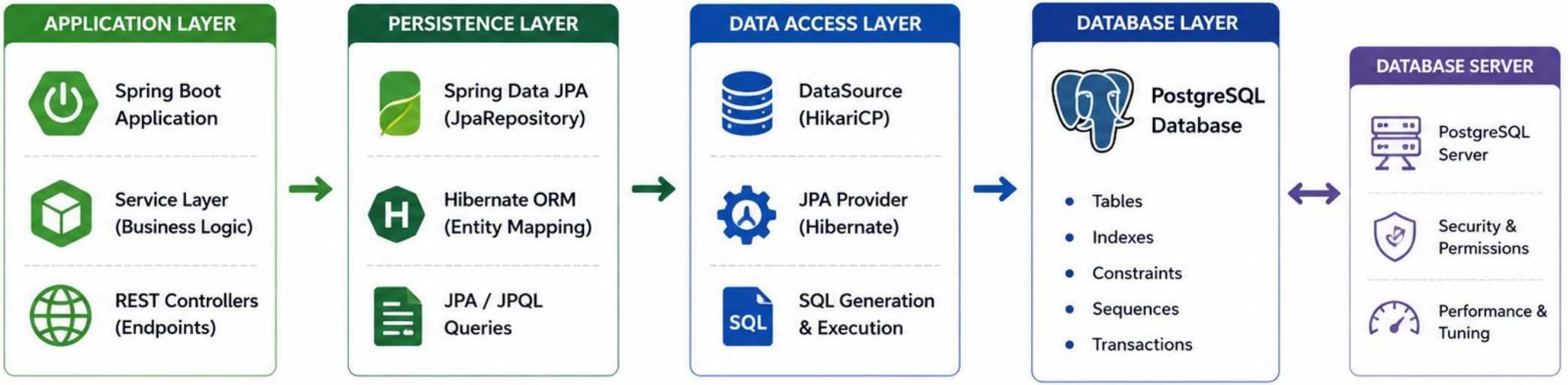
Wiring Spring Data JPA to PostgreSQL specifically needs three pieces: a driver, a DataSource, and dialect-aware configuration.

Piece	Role
JDBC Driver (org.postgresql:postgresql)	Speaks PostgreSQL's wire protocol -- covered in Module 37.
DataSource	Manages a pool of open PostgreSQL connections for the application to reuse.
Hibernate PostgreSQL Dialect	Translates JPA operations into PostgreSQL-specific SQL syntax.
application.properties	Ties the three together -- URL, credentials, and JPA behavior flags.

KEY TAKEAWAY The same PostgreSQL JDBC driver from Module 37 sits underneath everything -- Spring Boot just adds a managed connection pool and Hibernate's PostgreSQL dialect on top of it.

PostgreSQL Integration Architecture

Spring Boot applications use Spring Data JPA with Hibernate as the ORM layer to seamlessly interact with PostgreSQL as the relational database.



INTEGRATION FLOW

1. Application calls Service or Repository → 2. Spring Data JPA processes the request → 3. Hibernate converts to SQL (or uses native SQL) → 4. SQL executed against PostgreSQL → 5. Results mapped back to Entities/DTOs and returned

KEY BENEFITS

- Strong Data Integrity with ACID compliance
- High Performance and Scalability
- Advanced PostgreSQL Features (JSONB, CTEs, Indexes, Extensions)
- Security, Roles & Least Privilege
- Reliable Transactions and Concurrency Control

ADDING POSTGRESQL DRIVER DEPENDENCY

The same org.postgresql:postgresql dependency from Module 37, now added to a Spring Boot project.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

KEY TAKEAWAY spring-boot-starter-data-jpa brings in Spring Data JPA and Hibernate together; the PostgreSQL driver, exactly as in Module 37, stays scope=runtime.

Adding PostgreSQL Driver Dependency

Spring Boot uses JDBC to connect to PostgreSQL. Add the PostgreSQL driver dependency so your application can communicate with the database.

WHERE TO ADD

Add the PostgreSQL driver dependency in your project's build file.

Maven
pom.xml

Gradle
build.gradle

MAVEN (pom.xml)

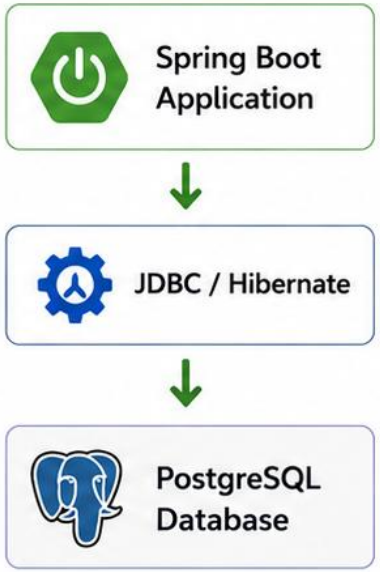
```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.3</version>
    <scope>runtime</scope>
</dependency>
```

The <scope>runtime</scope> means the driver is required at runtime, not during compilation.

GRADLE (build.gradle)

```
dependencies {
  runtimeOnly 'org.postgresql:postgresql:42.7.3'
}
```

Use runtimeOnly so the driver is included in the runtime classpath but not in compile classpath.



ABOUT THE POSTGRESQL DRIVER

The PostgreSQL JDBC driver enables Java applications to connect to PostgreSQL databases using standard JDBC APIs. It supports all PostgreSQL features including SSL, JSONB, and advanced data types.

BEST PRACTICES

- Always use the latest stable driver version.
- Use runtime scope to reduce build dependencies.
- Verify driver compatibility with your PostgreSQL version.
- Keep dependencies updated for security and performance.

DATASOURCE CONFIGURATION

Spring Boot auto-configures a connection-pooled DataSource from a handful of properties -- no manual bean definition required.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/crmdb
spring.datasource.username=crm_app
spring.datasource.password=${CRM_DB_PASSWORD}
spring.datasource.hikari.maximum-pool-size=10
```

KEY TAKEAWAY The exact jdbc:postgresql:// URL format from Module 37, plus HikariCP's pool size -- Spring Boot wires the whole DataSource bean automatically from these properties.

DataSource Configuration

The DataSource is the connection factory that provides JDBC connections from a pool to your Spring Boot application.

ROLE OF DATASOURCE

- Manages database connections
- Provides connection pooling for performance
- Ensures secure and reliable connectivity
- Configured automatically or manually in Spring Boot

SPRING BOOT DATASOURCE BEAN

```
@Configuration
public class DataSourceConfig {

    @Bean
    @ConfigurationProperties(prefix = "app.datasource")
    public DataSource dataSource() {
      return DataSourceBuilder.create().build();
    }
}
```

Spring Boot auto-configures a DataSource if the required properties are defined in application.properties or application.yml.

DATASOURCE PROPERTIES (application.properties)

```
app.datasource.url=jdbc:postgresql://localhost:5432/employee_db
app.datasource.username=etl_user
app.datasource.password=etl_pass
app.datasource.driver-class-name=org.postgresql.Driver
app.datasource.hikari.minimum-idle=5
app.datasource.hikari.maximum-pool-size=20
app.datasource.hikari.idle-timeout=30000
app.datasource.hikari.connection-timeout=20000
```

COMMON PROPERTIES

Property	Description
url	JDBC connection URL to PostgreSQL
username / password	Database credentials
driver-class-name	JDBC driver class
hikari.minimum-idle	Minimum idle connections in the pool
hikari.maximum-pool-size	Maximum connections in the pool
hikari.connection-timeout	Time to wait for a connection from the pool

BEST PRACTICES

- Use connection pooling (HikariCP) for performance.
- Store credentials securely (avoid hardcoding).
- Set appropriate pool sizes based on workload.
- Validate connections and handle timeouts properly.

APPLICATION.PROPERTIES CONFIGURATION

A handful of JPA-specific properties control exactly how Hibernate behaves against PostgreSQL.

Property	Recommended Value & Why
spring.jpa.hibernate.ddl-auto	validate -- Hibernate checks the schema matches; it never mutates it. Flyway owns schema changes.
spring.jpa.open-in-view	false -- forces lazy data access to happen inside the transaction, not accidentally in the view layer.
spring.jpa.show-sql	true in development -- prints every generated SQL statement, invaluable for the Module 38 diagnostic habits.
spring.jpa.properties.hibernate.dialect	Usually auto-detected -- PostgreSQL dialect for correct, database-specific SQL generation.

KEY TAKEAWAY ddl-auto: validate and open-in-view: false are the two settings every production Spring Boot + PostgreSQL project should set deliberately -- both prevent a class of real bugs.

TRY IT YOURSELF — EXERCISE 1: DECIDE WHO OWNS THE SCHEMA

Reinforces: the persistence architecture, JPA and Hibernate overview, and datasource configuration slides you just covered.

STEPS (notes/lab39-schema-owner.md)

Step	What To Do
1 — Layer Roles	Say what JPA, Hibernate, and Spring Data JPA each contribute.
2 — Datasource Keys	Name the properties that configure the PostgreSQL datasource.
3 — Schema Owner	State who owns the schema and what ddl-auto is set to.
4 — No Secrets	Say where the datasource password comes from at run time.

Configuration shape

```
spring.datasource.url=jdbc:postgresql://localhost:5432/crm
spring.jpa.hibernate.ddl-auto=validate    # migrations own the schema
password: injected from the environment -- never in application.yml
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-datasource-and-schema-owner.md (workspace: examples/module-39-exercises).

`application.properties` Configuration

Spring Boot uses `application.properties` to configure the connection to PostgreSQL and other application settings.

WHERE TO CONFIGURE

src/main/resources/
application.properties

Spring Boot automatically loads this file at startup.

KEY BENEFITS

- Centralized configuration
- Environment-specific customization
- Externalized configuration for flexibility
- Easy to manage and update without code changes

EXAMPLE: application.properties

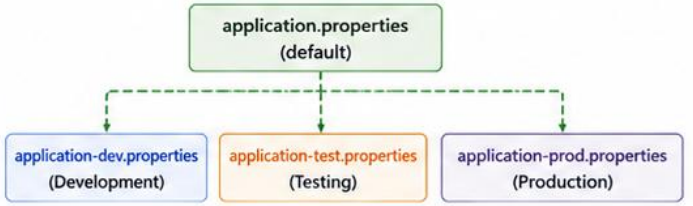
```
# Server Configuration
server.port=8080

# PostgreSQL DataSource Configuration
spring.datasource.url=jdbc:postgresql://localhost:5432/employee_db
spring.datasource.username=etl_user
spring.datasource.password=etl_pass
spring.datasource.driver-class-name=org.postgresql.Driver
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.idle-timeout=30000
spring.datasource.hikari.connection-timeout=20000

# JPA / Hibernate Configuration
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

COMMON PROPERTY CATEGORIES

- **Server**
  Port, context path, error settings
- **DataSource**
  Database URL, credentials, pool settings
- **JPA / Hibernate**
  Dialect, DDL auto, SQL logging
- **Logging**
  Log levels and output configuration
- **Security**
  JWT, OAuth, and other security settings

ENVIRONMENT SPECIFIC CONFIGURATION

Override properties for each environment using profile files.



BEST PRACTICES

- Store sensitive values in environment variables or secrets, not in code.
- Use connection pooling (HikariCP) for better performance.
- Set appropriate timeouts and pool sizes based on workload.
- Use profiles to manage different environments cleanly.
- Never commit real credentials to source control.

ENTITY CLASSES

An entity is a plain Java class, annotated with @Entity, where each instance represents one row of a PostgreSQL table.

```
@Entity
@Table(name = "customer")
public class CustomerEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "customer_id")
    private Long id;

    @Column(name = "full_name", nullable = false)
    private String fullName;

    // no-arg constructor, getters/setters
}
```

KEY TAKEAWAY Every entity needs @Entity, a mapped primary key via @Id, and a no-arg constructor -- this one class is the direct Java counterpart of Module 37's customer table.

Entity Classes

Entity classes represent database tables in a relational database.
They are annotated with JPA annotations and managed by Hibernate.

KEY POINTS

- Each entity maps to one database table.
- A primary key uniquely identifies each record.
- Fields map to table columns.
- Relationships map to foreign keys.
- Entities are managed in the persistence context.

EXAMPLE: EMPLOYEE ENTITY

```
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")                                          // Primary Key
    private Long id;

    @Column(name = "first_name", nullable = false, length = 50)   // Column Mapping
    private String firstName;

    @Column(name = "last_name", nullable = false, length = 50)    // Column Mapping
    private String lastName;

    @Column(name = "email", unique = true, nullable = false, length = 100)   // Unique
    private String email;                                                      Column

    @Column(name = "salary", precision = 10, scale = 2)
    private BigDecimal salary;                                    // Numeric Column

    @Column(name = "hire_date")
    private LocalDate hireDate;                                   // Date Column
}
```

MAPPING OVERVIEW

Annotation	Purpose
@Entity	Marks the class as a JPA entity.
@Table	Specifies the table name.
@Id	Defines the primary key.
@GeneratedValue	Auto-generates primary key values.
@Column	Maps the field to a table column.

DATABASE TABLE (employee)

Column Name	Data Type	Constraints
id	BIGINT	PK, Auto Increment
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
salary	NUMERIC(10,2)	NULL
hire_date	DATE	NULL

BEST PRACTICES

- Keep entities focused and aligned with business concepts.
- Use meaningful field names and consistent naming conventions.
- Avoid exposing entity classes directly in API responses.
- Use DTOs to decouple persistence layer from API contracts.
- Keep entities small, cohesive, and easy to maintain.

@ENTITY

@Entity is the single annotation that turns a plain Java class into a JPA-managed persistent class.

Registers the class with the JPA provider (Hibernate) at application startup.
Without @Table, the entity maps to a table with the same name as the class -- rarely what you want with PostgreSQL's snake_case convention, so @Table is used almost universally.
The class must not be final, and must have a no-arg constructor Hibernate can call.
An @Entity class is a plain Java object otherwise -- ordinary fields, getters, and methods.

KEY TAKEAWAY @Entity is the minimum annotation needed to make Hibernate aware of a class -- everything else covered this module refines how that class maps, not whether it maps at all.

`@Entity`

Marking a Java Class s a JPA Entity

What is `@Entity`?

`@Entity` is a JPA annotation that marks a class as a persistent entity. The class will be mapped to a database table by the JPA provider (e.g., Hibernate).

Key Points

- Marks a class as a JPA entity.
- The class must have a primary key (using `@Id`).
- Each entity instance maps to a row in a database table.
- The table name defaults to the class name (can be overridden with `@Table`).
- Requires a no-argument constructor.

Best Practice

Keep entity classes focused on data. Avoid business logic and external service calls inside entities.

Example: Entity Class

```java
import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String firstName;

    @Column(nullable = false, length = 100)
    private String lastName;

    @Column(nullable = false, unique = true)
    private String email;

    // No-argument constructor required by JPA
    public Employee() {}
}
```

How It Maps

Employee (Entity Class)

employees (Table)

Column	Type
id (PK)	bigint
first_name	varchar(100)
last_name	varchar(100)
email (Unique)	varchar(100)
...	...

Remember

`@Entity` is the foundation of ORM in JPA. It tells the framework: "This class is a persistent entity—manage it and map it to a database table."

@TABLE

@Table names the PostgreSQL table an entity maps to -- and can specify more than just the name.

```
@Entity
@Table(name = "customer", schema = "public",
       uniqueConstraints = @UniqueConstraint(columnNames = "email"))
public class CustomerEntity { ... }
```

KEY TAKEAWAY @Table's name attribute is the one used constantly; schema and uniqueConstraints exist for the less common cases where the default public schema or Module 37's own DDL constraints aren't already sufficient.

@Table

Mapping Entity to a Database Table

Module 39
Spring Data JPA and PostgreSQL Integration

Purpose

The `@Table` annotation specifies the database table that an entity maps to. It allows customizing the table name, schema, and other table-level attributes.

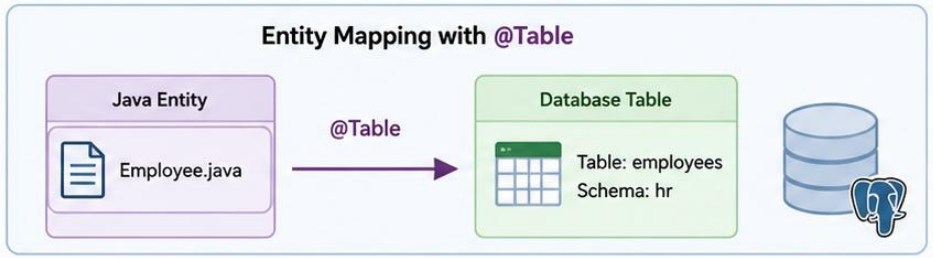
Common Attributes

- **`name`**: Name of the database table
- **`schema`**: Database schema (optional)
- **`catalog`**: Catalog name (optional)
- **`uniqueConstraints`**: Define table-level unique constraints

Key Takeaway

Use `@Table` to explicitly define the table name and schema, ensuring clarity and alignment with database naming standards.

```java
import jakarta.persistence.*;

@Entity
@Table(
    name = "employees",
    schema = "hr",
    uniqueConstraints = {
        @UniqueConstraints(name = "uk_employee_email",
        (       columnNames = "emaill")
    }
)
public class Employee {
    // fields, getters, setters
}
```



`@Table` bridges your Java entity classes with the actual database table structure.

@ID

@Id marks exactly one field as an entity's primary key -- JPA requires every entity to have one.

Every @Entity class must have exactly one field (or property) annotated @Id.
The type must be serializable and comparable -- Long, String, and UUID are all common choices.
@Id alone means the application supplies the value -- pair it with @GeneratedValue, covered next, for a database-generated value.
This is the direct entity-level counterpart of Module 37's PRIMARY KEY constraint.

KEY TAKEAWAY @Id is non-negotiable -- Hibernate cannot manage an entity without knowing which field uniquely identifies it, exactly mirroring why Module 37 called the primary key the anchor every relationship depends on.

@Id

Marks the field as the primary key of the entity.

Primary Key Identifier
The @Id annotation identifies the primary key field that uniquely represents each entity record.

Maps to Database Primary Key
This field is mapped to the table's primary key column in PostgreSQL.

Required in Every @Entity
Every JPA entity must have exactly one field annotated with @Id.

Works with ID Generation Strategies
Commonly used with @GeneratedValue to define how the primary key value is generated.

Employee.java

```java
import jakarta.persistence.*;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    // getters and setters
}
```

The @Id annotation marks this field as the primary key.

Key Takeaway
Use @Id to uniquely identify each entity. It forms the basis for relationships, joins, and data integrity in your application.

Entity Class → @Id → Table Primary Key → Unique Record Identity

IDENTITY GENERATION

@GeneratedValue tells Hibernate how a primary key's value is produced -- IDENTITY is the strong default for PostgreSQL.

Strategy	PostgreSQL Behavior
IDENTITY	Matches Module 37's GENERATED ALWAYS AS IDENTITY -- the database assigns the value on INSERT. The recommended default.
SEQUENCE	Uses a PostgreSQL sequence, fetched before insert -- better for high-volume batch inserts.
AUTO	Lets Hibernate pick a strategy based on the PostgreSQL dialect -- usually resolves to IDENTITY or SEQUENCE.

KEY TAKEAWAY GenerationType.IDENTITY is the direct Java-side counterpart of Module 37's GENERATED ALWAYS AS IDENTITY columns -- use it as the default unless a specific reason says otherwise.

MODULE 39

Identity Generation

Automatically generating unique primary key values for entities.

HIBERNATE

Spring Data JPA

What is Identity Generation?

Identity generation automatically assigns a **unique value to the primary key** field when a new entity is persisted. This ensures uniqueness without requiring the application to manually set the ID.

Common Identity Strategies

Strategy	Description
IDENTITY	Uses database auto-increment (e.g., SERIAL, IDENTITY column in PostgreSQL).
SEQUENCE	Uses a database sequence to generate unique values.
TABLE	Uses a table to simulate sequence generation.
UUID	Generates globally unique identifiers (UUIDs).

Best Practice

Use **IDENTITY** for simple database auto-increment needs.
Use **SEQUENCE** for high-performance and scalability.

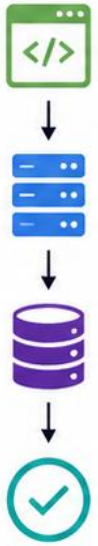
Example: Identity Generation with @GeneratedValue

```
@Entity
@Table(name = "employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    // getters and setters
}
```

Note

- With **IDENTITY**, the database generates the ID on insert.
- The ID value is available only after the entity is persisted.
- Commonly used with PostgreSQL **SERIAL** or **IDENTITY** columns.

How Identity Generation Works



1. Application
Saves a new entity without setting the ID.

2. JPA / Hibernate
Detects the ID is null and prepares the insert.

3. Database
Generates a unique ID using auto-increment / identity mechanism.

4. Persisted Entity
The generated ID is returned and set on the entity.

Example (PostgreSQL)

```
CREATE TABLE employee (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);
```

Key Takeaway: Identity generation simplifies primary key management and ensures unique, reliable identifiers for your entities.

@COLUMN

@Column customizes exactly how a field maps to its PostgreSQL column -- name, nullability, length, and uniqueness.

```
@Column(name = "full_name", nullable = false, length = 150, unique = false)
private String fullName;
```

nullable = false is the entity-side mirror of Module 37's NOT NULL constraint.
length applies to VARCHAR columns and should match the DDL's declared length exactly.
Without @Column at all, the field name itself becomes the column name.

KEY TAKEAWAY Every attribute on @Column -- nullable, length, unique -- is the entity-side mirror of a constraint Module 37 already taught at the DDL level; the two should always agree.

@Column

The **@Column** annotation is used to map an entity field to a table column and define column-level properties.

Key Points

- Maps a Java field to a specific column in the database table.
- Allows customization of column name, length, nullability, uniqueness, and other constraints.
- If name is not specified, the field name is used as the column name.
- Works with basic types, wrapper types, String, and more.
- Commonly used with **@Entity** fields to control schema mapping.

Best Practice

Always define column constraints explicitly using **@Column** to ensure data integrity and backward-compatible schema evolution.

Example

```java
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", length = 50, nullable = false)
    private String firstName;

    @Column(name = "email", unique = true, nullable = false)
    private String email;
}
```

Mapping Result

Employee Entity

id: Long

firstName: String

email: String

employees (Table)

Column Name	Data Type	Constraints
id	BIGSERIAL	PRIMARY KEY
first_name	VARCHAR(50)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE

ENTITY LIFECYCLE

Every entity instance moves through four states -- Transient, Managed, Detached, and Removed -- and Hibernate tracks which one it's in.

State	Meaning
Transient	A plain new object -- not yet associated with Hibernate at all.
Managed	Tracked by the persistence context -- changes are detected automatically.
Detached	Was managed, no longer tracked -- changes are silently ignored until re-attached.
Removed	Marked for deletion -- will be deleted on the next flush or commit.

KEY TAKEAWAY A Managed entity's field changes are picked up automatically by Hibernate's dirty checking; a Detached entity's are not -- this single distinction explains a large share of real JPA bugs.

TRY IT YOURSELF — EXERCISE 2: MAP THE CUSTOMER ENTITY

Reinforces: entity classes, @Entity, @Table, @Id, identity generation, @Column, and the entity lifecycle you just covered.

STEPS (notes/lab39-entity-mapping.md)

Step	What To Do
1 — Annotations	List the annotations on CustomerEntity and what each one declares.
2 — Id Strategy	Choose the identity generation strategy and say why it suits PostgreSQL.
3 — Column Mapping	Show one @Column with an explicit name, nullability, and length.
4 — Lifecycle	Name the entity states and which one holds a database identity.

Entity sketch

```
@Entity @Table(name="customer")  class CustomerEntity
  @Id @GeneratedValue(strategy=IDENTITY) private Long id;
  @Column(name="email", nullable=false, length=255) private String email;
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-entity-mapping.md (workspace: examples/module-39-exercises).

MODULE 39

Entity Lifecycle

The different states an entity goes through from creation to removal within a persistence context.

HIBERNATE

Spring Data JPA

What is an Entity Lifecycle?

An entity changes state as it is managed by the EntityManager within a persistence context. Understanding the lifecycle helps in writing efficient, bug-free, and performant data access code.

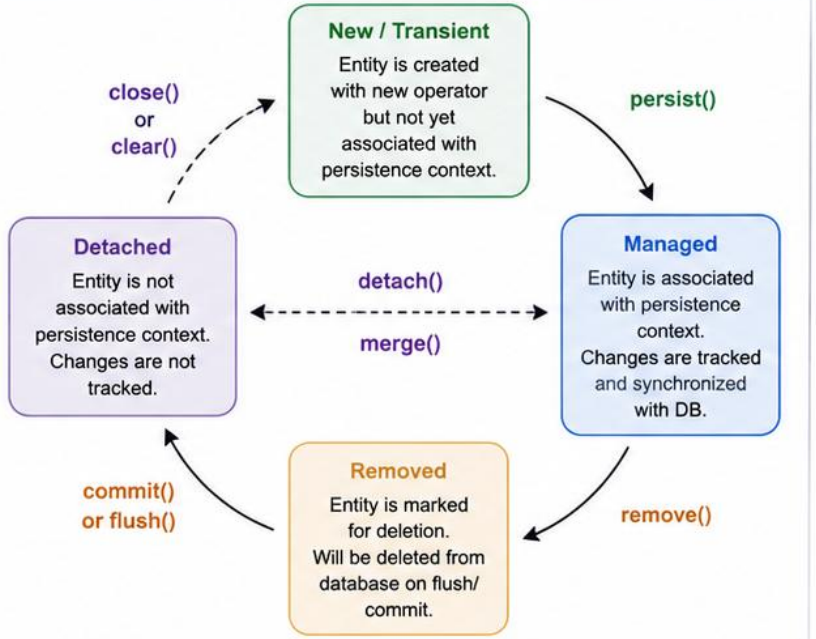
Key Points

- An entity is always in one of the lifecycle states at any given time.
- State transitions are managed by the EntityManager.
- Only managed entities are synchronized with the database.
- Detached entities are not tracked until reattached.
- Removed entities are deleted on flush/commit.

Best Practices

- Keep transactions short and focused.
- Avoid holding references to detached entities for long periods.
- Use fetch strategies wisely to avoid N+1 queries.

Entity Lifecycle States and Transitions



Remember

Only managed entities are tracked and synchronized with the database. Always be aware of the entity state when performing operations.

Lifecycle States Summary

State	Description	Tracked?	In Sync?
New / Transient	Created but not persisted.	No	No
Managed	Associated with EntityManager.	Yes	Yes
Detached	Not associated with EntityManager.	No	No
Removed	Marked for deletion.	Yes	Yes (until delete)

Example Code

```
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();

tx.begin();
Employee e = new Employee("John", "Developer");
em.persist(e);                  // New -> Managed
e.setName("John Smith");        // Managed (dirty)
em.detach(e);                   // Managed -> Detached
e.setName("Johnny");            // No effect in DB
e = em.merge(e);                // Detached -> Managed
em.remove(e);                   // Managed -> Removed
tx.commit();                    // Removed -> Detached
em.close();
```

Key Takeaway: Understanding the entity lifecycle helps you write efficient JPA applications, avoid common bugs, and make the most of persistence context management.

REPOSITORY INTERFACES

A repository is an interface -- Spring Data JPA generates the implementing class automatically at startup.

Interface	Adds
Repository<T, ID>	The base marker interface -- adds nothing on its own.
CrudRepository<T, ID>	save, findById, findAll, deleteById, and more basic CRUD.
PagingAndSortingRepository<T, ID>	Adds Pageable and Sort support.
JpaRepository<T, ID>	Adds JPA-specific batch and flush methods -- the interface this course extends.

KEY TAKEAWAY This course extends JpaRepository for every entity -- it includes everything the three interfaces below it provide, plus JPA-specific extras.

MODULE 39

Repository Interfaces

Spring Data JPA provides repository interfaces that give you powerful data access capabilities with minimal boilerplate.

HIBERNATE

Spring Data JPA

What is a Repository Interface?

A repository interface is a contract for data access. Spring Data JPA automatically provides the implementation at runtime.

You focus on defining methods – **Spring Data JPA** takes care of the rest!

Key Points

- Interfaces extend **JpaRepository** (or similar).
- Provides CRUD methods out of the box.
- Supports derived query methods.
- Supports custom queries (JPQL or native SQL).
- Fully integrated with transactions and entity management.
- Type-safe and testable.

Best Practice

- Keep repositories focused on data access.
- Use meaningful method names for derived queries.
- Prefer derived queries; use `@Query` only when necessary.

Example: EmployeeRepository Interface

```
package com.example.employee.repository;

import com.example.employee.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.streeotype.Repository;

@Repository
public interface EmployeeRepository
      extends JpaRepository<Employee, Long>,
              JpaSpecificationExecutor<Employee> {

   // Derived query methods
   Optional<Employee> findByEmail(String email);
   List<Employee> findByDepartment(String department);
   List<Employee> findByFirstNameContainingIgnoreCase(String name);
   boolean existsByEmail(String email);

   // Custom query with JPQL
   @Query("select e from Employee e where e.active = true order by e.createdAt desc")
   List<Employee> findActiveEmployeesOrderByCreatedAtDesc();

   // Custom native query
   @Query(value = "SELECT * FROM employees WHERE salary > :salary", nativeQuery = true)
   List<Employee> findBySalaryGreaterThan(@Param("salary") BigDecimal salary);
}
```

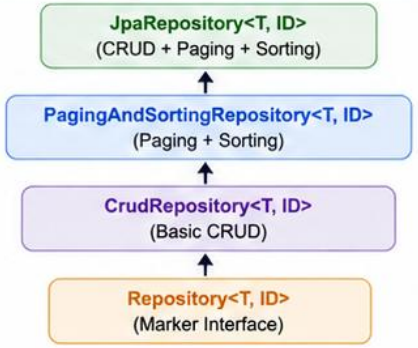
Remember

Spring Data JPA generates the implementation of repository interfaces at runtime. You only define the contract!

CRUD Methods You Get Automatically

Method	Description
save(entity)	Insert or update an entity
findById(id)	Find entity by primary key
existsById(id)	Check if entity exists
findAll()	Find all entities
findAll(Sort)	Find all entities with sorting
deleteById(id)	Delete entity by ID
delete(entity)	Delete the given entity
count()	Count total entities

Repository Inheritance Hierarchy



Key Takeaway: Repository interfaces simplify data access, reduce boilerplate, and let you focus on business logic. Leverage derived queries and custom queries to build powerful, maintainable applications.

JPAREPOSITORY

Extending JpaRepository<CustomerEntity, Long> with an empty body already provides a complete, working data access layer.

```
public interface CustomerRepository extends JpaRepository<CustomerEntity, Long> {
    // empty -- save, findById, findAll, deleteById, and more, all included
}
```

Spring Data JPA generates the implementing class at application startup -- no code to write.
Available immediately: save, saveAll, findById, findAll, existsById, count, deleteById, delete, and Pageable/Sort-aware overloads.
Custom methods, covered next, are simply added as additional interface method declarations.

KEY TAKEAWAY An interface with zero method bodies already provides a complete data access layer -- this is Spring Data JPA's core productivity promise, made concrete.

MODULE 39

JpaRepository

JpaRepository is a core Spring Data JPA interface that provides powerful CRUD, paging, and sorting capabilities out of the box.

HIBERNATE

Spring Data JPA

What is JpaRepository?

JpaRepository<T, ID> is a Spring Data JPA interface that extends PagingAndSortingRepository and CrudRepository.

It provides a complete set of CRUD operations along with pagination and sorting support.

Key Benefits

- No boilerplate code for common data access.
- Type-safe and easy to extend.
- Built-in CRUD methods.
- Supports pagination and sorting.
- Easily create derived queries or custom queries.

Best Practices

- Use meaningful repository names.
- Prefer derived queries for simple use cases.
- Use @Query for complex JPQL or native queries.
- Keep repositories focused on a single aggregate root (entity).

Example: Using JpaRepository

```
@Repository
public interface EmployeeRepository
      extends JpaRepository<Employee, Long> {

  // Derived query methods
  List<Employee> findByDepartment(String department);
  List<Employee> findByFirstNameContainingIgnoreCase(String name);
  Optional<Employee> findByEmail(String email);
  boolean existsByEmail(String email);
  long countByDepartment(String department);

  // Custom JPQL query
  @Query("select e from Employee e where e.active = true order by e.createdAt desc")
  List<Employee> findActiveEmployeesOrderByCreatedAtDesc();

  // Custom native query
  @Query(value = "SELECT * FROM employees WHERE salary > :salary", nativeQuery = true)
  List<Employee> findBySalaryGreaterThan(@Param("salary") BigDecimal salary);
}
```

Your Repository
EmployeeRepository extends JpaRepository<Employee, Long>
(Your custom methods)

→

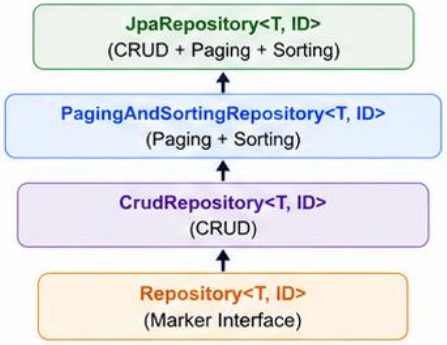
Spring Data JPA
Generates implementation at runtime

→

Database
Execute queries

Note
JpaRepository reduces development time and ensures consistency across all repositories.

JpaRepository Inheritance Hierarchy



Common CRUD Methods

Method	Description
save(entity)	Insert or update an entity
findById(id)	Find entity by ID
existsById(id)	Check if entity exists
findAll()	Find all entities
findAll(Sort sort)	Find all with sorting
findAll(Pageable pageable)	Find all with pagination
deleteById(id)	Delete entity by ID
delete(entity)	Delete entity
count()	Count total entities

Key Takeaway: JpaRepository gives you a rich set of features with minimal code. Focus on your business logic, not boilerplate data access.

CRUD OPERATIONS

JpaRepository's four core operations map directly onto the entity lifecycle states covered earlier this module.

Operation	Method	What Happens
Create	save(entity)	Transient -> Managed; Hibernate issues an INSERT.
Read	findById(id) / findAll()	Loads row(s) into Managed entities.
Update	save(entity)	Managed entity's changes are flushed as an UPDATE.
Delete	deleteById(id) / delete(entity)	Managed -> Removed; Hibernate issues a DELETE

KEY TAKEAWAY save() does double duty for both Create and Update -- Hibernate decides which SQL to generate based on whether the entity's ID is already populated.

MODULE 39

CRUD Operations

Spring Data JPA with JpaRepository provides built-in methods for Create, Read, Update, and Delete operations.

HIBERNATE

Spring Data JPA

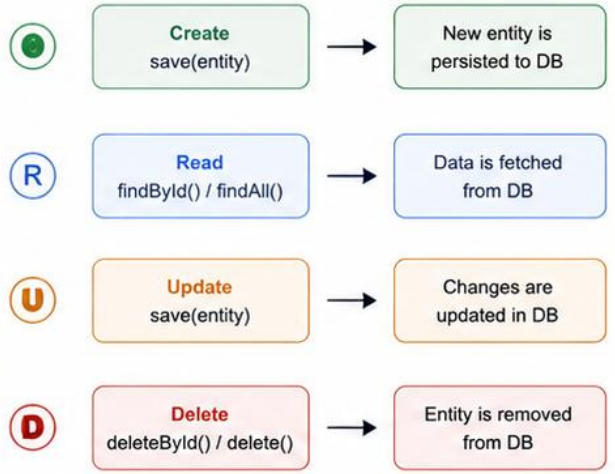
Built-in CRUD Methods in JpaRepository

Operation	Method	Description	Returns
Create	save(entity)	Inserts a new entity or updates if ID exists	T
Read (By ID)	findById(id)	Finds an entity by primary key	Optional<T>
Read (All)	findAll()	Retrieves all entities	List<T>
Update	save(entity)	Updates an existing entity	T
Delete (By ID)	deleteById(id)	Deletes entity by primary key	void
Delete (Entity)	delete(entiity)	Deletes the given entity	void
Exists	existsById(id)	Checks if an entity exists by ID	boolean
Count	count()	Counts total entities	long

Example: Employee CRUD with JpaRepository

```
@Autowired
private EmployeeRepository employeeRepository;

// CREATE
Employee e = new Employee("John", "Developer", "IT");
Employee saved = employeeRepository.save(e);

// READ - BY ID
Optional<Employee> empOpt = employeeRepository.findById(saved.getId());
empOpt.ifPresent(emp -> System.out.println(emp));

// READ - ALL
List<Employee> allEmployees = employeeRepository.findAll();

// UPDATE
e.setName("John Smith");
Employee updated = employeeRepository.save(e);

// DELETE - BY ID
employeeRepository.deleteById(saved.getId());

// DELETE - ENTITY
Employee emp = employeeRepository.findById(1L).orElseThrow();
employeeRepository.delete(emp);

// EXISTS & COUNT
boolean exists = employeeRepository.existsById(1L);
long total = employeeRepository.count();
```

CRUD Flow



Key Takeaways

- JpaRepository eliminates boilerplate CRUD code.
- save() handles both insert and update.
- findById() returns Optional<T> to handle missing data safely.
- deleteById() is more efficient than fetching and deleting.
- Use meaningful repository names for clarity.

Key Takeaway: JpaRepository provides powerful CRUD capabilities out of the box, helping you build robust and maintainable data access layers with minimal code.

DERIVED QUERY METHODS

Spring Data JPA parses a correctly-shaped method name into a real query -- no @Query annotation required for common cases.

Method Name Keyword	Translates To
findBy<Field>	WHERE field = ?
And / Or	WHERE a = ? AND/OR b = ?
GreaterThan / LessThan	WHERE field > ? / field < ?
Containing	WHERE field LIKE %?%
OrderBy<Field>Desc/Asc	ORDER BY field DESC/ASC

KEY TAKEAWAY Introduce derived queries as the single most-used Spring Data JPA feature after basic CRUD -- write a correctly-shaped method NAME, and the query is generated automatically.

MODULE 39

Derived Query Methods

Spring Data JPA can generate query implementations automatically based on method names. No JPQL or SQL required!

HIBERNATE

Spring Data JPA

What are Derived Query Methods?

Derived query methods are created by defining method names following a specific naming convention.

Spring Data JPA parses the method name and generates the query automatically at runtime.

Key Benefits

- No need to write JPQL or SQL for common queries.
- Reduces boilerplate code.
- Improves productivity and readability.
- Type-safe and refactor-friendly.
- Supports complex queries with method name keywords.

Naming Convention

findBy + Field + Condition + (And / Or) + (OrderBy / IgnoreCase / etc.)

Example:

findByDepartmentAndSalaryGreaterThan

Example: Employee Repository with Derived Query Methods

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

  // Simple conditions
  List<Employee> findByDepartment(String department);
  Optional<Employee> findByEmail(String email);
  List<Employee> findByActiveTrue();

  // Multiple conditions
  List<Employee> findByDepartmentAndSalaryGreaterThan(String department, BigDecimal salary);
  List<Employee> findByFirstNameContainingIgnoreCase(String name);

  // Sorting
  List<Employee> findByDepartmentOrderByLastNameAsc(String department);

  // Top / First
  Optional<Employee> findTopByOrderByCreatedAtDesc();
  List<Employee> findFirst10ByActiveTrueOrderBySalaryDesc();
}
```

Examples and Generated Queries

Derived Method	Generated JPQL (Simplified)	Description
findByDepartment(String dept)	select e from Employee e where e.department = :dept	Find by department
findByEmail(String email)	select e from Employee e where e.email = :email	Find by email
findByActiveTrue()	select e from Employee e where e.active = true	Find all active employees
findByDepartmentAndSalaryGreaterThan (String dept, BigDecimal salary)	select e from Employee e where e.department = :dept and e.salary > :salary	Multiple conditions (AND)
findByFirstNameContainingIgnoreCase (String name)	select e from Employee e where lower(e.firstName) like lower(concat('%', :name, '%'))	Contains (case-insensitive)
findByDepartmentOrderByLastNameAsc (String dept)	select e from Employee e where e.department = :dept order by e.lastName asc	Sorted results
findTopByOrderByCreatedAtDesc()	select e from Employee e order by e.createdAt desc limit 1	Top 1 latest record

Common Keywords

Keyword	Meaning
And	Combine conditions using AND
Or	Combine conditions using OR
Between	Between a range
LessThan / GreaterThan	Comparison operators
Like / Containing	Pattern matching
IgnoreCase	Case-insensitive match
OrderBy	Sort results
Top / First	Limit number of results
IsNull / IsNotNull	Null checks
True / False	Boolean conditions

Best Practices

- Use meaningful method names.
- Keep methods focused on a single responsibility.
- Use @Query for complex queries that are hard to express with method names.
- Always write tests for repository methods.

Key Takeaway: Derived query methods let Spring Data JPA build queries for you by reading method names, making your code cleaner, faster to develop, and easier to maintain.

FIND BY METHODS

findBy is the most common derived query prefix -- worked examples against the Northstar CRM entities.

```
public interface CustomerRepository extends JpaRepository<CustomerEntity, Long> {

    Optional<CustomerEntity> findByPublicId(String publicId);

    List<CustomerEntity> findByFullNameContaining(String namePart);

    List<CustomerEntity> findByStatusOrderByFullNameAsc(String status);
}
```

KEY TAKEAWAY Return type signals cardinality to Spring Data JPA: Optional<T> for at-most-one match, List<T> for many -- picking the wrong one either compiles incorrectly or throws at runtime.

Find By Methods

Spring Data JPA allows us to create query methods by following a naming convention.
The framework automatically generates the query at runtime.

Example Entity

```
@Entity
@Table(name = "employees")
public class Employee {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String firstName;
  private String lastName;
  private String email;
  private String department;
  private Boolean active;
  private LocalDate hireDate;
}
```

Repository Interface with Find By Methods

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

   Optional<Employee> findById(Long id);
   List<Employee> findByLastName(String lastName);
   List<Employee> findByDepartment(String department);
   List<Employee> findByActiveTrue();
   List<Employee> findByHireDateAfter(LocalDate date);
   List<Employee> findByDepartmentAndActiveTrue(String department);
   Optional<Employee> findByEmail(String email);
   List<Employee> findByLastNameContainingIgnoreCase(String keyword);

}
```

Naming Convention

Method names are parsed based on keywords. Examples:

Keyword	Meaning
findBy	Starts a query based on a field
And	Combines multiple conditions
Or	Alternative conditions
True / False	Matches boolean fields
After / Before	Date comparison
Containing	LIKE %value% match
IgnoreCase	Case-insensitive match

Benefits

Faster Development
No need to write JPQL or SQL for common queries.

Type Safe
Checked at compile time, avoids runtime errors.

Readable & Maintainable
Self-explanatory method names improve code clarity.

Tip

Use derived query methods for simple queries. For complex queries, use `@Query` with JPQL or native SQL.

MULTIPLE CRITERIA QUERIES

Chaining And/Or in a derived query name works well for two or three conditions -- beyond that, readability suffers.

```
List<CustomerEntity> findByStatusAndCreatedAtBetween(
    String status, Instant from, Instant to);

List<CustomerEntity> findByStatusOrFullNameContainingIgnoreCase(
    String status, String namePart);
```

KEY TAKEAWAY A derived method name with three or more chained conditions is a strong, concrete signal it's time to switch to @Query with JPQL instead.

MULTIPLE CRITERIA QUERIES

Spring Data JPA allows you to build queries using multiple criteria by combining keywords in method names. The framework generates the query automatically.

Spring Data JPA

Example Use Case

Find all active employees in a department with salary greater than a given amount.

Entity: Employee

- **id** : Long (Primary Key)
- **firstName** : String
- **lastName** : String
- **department** : String
- **salary** : BigDecimal
- **active** : Boolean
- **hireDate** : LocalDate

Repository Interface with Multiple Criteria Methods

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

 List<Employee> findByDepartmentAndActiveTrue(String department);

 List<Employee> findByDepartmentAndSalaryGreaterThan(
      String department, BigDecimal salary);

 List<Employee> findByDepartmentAndActiveTrueAndSalaryGreaterThan(
      String department, BigDecimal salary);

 List<Employee> findByActiveTrueAndHireDateBefore(LocalDate date);

 List<Employee> findByDepartmentAndActiveTrueAndHireDateBetween(
      String department, LocalDate startDate, LocalDate endDate);
}
```

Generated SQL Example

```
SELECT e.*
FROM employees e
WHERE e.department = ?1
      AND e.active = true
      AND e.salary > ?2;
```

?1 -> department (String)

?2 -> salary (BigDecimal)

Common Keywords for Multiple Criteria

Keyword	Description
And	Combine multiple conditions (AND)
Or	Combine alternative conditions (OR)
True / False	Match boolean fields
GreaterThan	Greater than value
LessThan	Less than value
Between	Range between two values
Before	Before a date/time
After	After a date/time
Like	Pattern match (e.g., %John%)
IgnoreCase	Case-insensitive match

Benefits

Concise & Expressive
Combine multiple filters in the method name—no JPQL needed.

Type Safe
Checked at compile time, reduces runtime errors.

Improves Productivity
Faster development with auto-generated queries.

Tip

For complex queries beyond method naming, use **@Query** with JPQL or native SQL.

JPQL

JPQL (Java Persistence Query Language) queries entity classes and their fields -- never tables and columns -- and stays database independent.

```
@Query("SELECT c FROM CustomerEntity c "
     + "WHERE c.status = :status ORDER BY c.fullName ASC")
List<CustomerEntity> findStatusOrdered(@Param("status") String status);
```

KEY TAKEAWAY JPQL looks like SQL but operates on entities and fields, never tables and columns -- CustomerEntity and c.status, never customer and status.

JPQL

JPQL (Java Persistence Query Language) is an object-oriented query language used with JPA to query entities and their relationships.

What is JPQL?

- JPQL works with entity classes and their fields, not database tables and columns.
- Portable across different databases.
- Supports object navigation, joins, aggregations, and parameters.

Example Entity: Employee

```
@Entity
@Table(name = "employees")
public class Employee {

 @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 private String firstName;
 private String lastName;
 private String department;
 private BigDecimal salary;
 private Boolean active;
}
```

Repository Interface Using JPQL

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Long> {

  @Query("SELECT e FROM Employee e WHERE e.department = :dept")
  List<Employee> findByDepartment(@Param("dept") String department);

  @Query("SELECT e FROM Employee e WHERE e.salary > :minSalary")
  List<Employee> findBySalaryGreaterThan(@Param("minSalary") BigDecimal minSalary);

  @Query("SELECT e FROM Employee e WHERE e.active = true
       AND e.department = :dept ORDER BY e.lastName")
  List<Employee> findActiveByDepartmentOrderByLastName(@Param("dept") String department);

  @Query("SELECT e FROM Employee e WHERE e.lastName LIKE :name")
  List<Employee> searchByLastName(@Param("name") String name);
}
```

JPQL Example

JPQL Query	Explanation

`SELECT e FROM Employee e`
`WHERE e.department = :dept`
`  AND e.salary > :minSalary`
`ORDER BY e.lastName`
	• **SELECT** e – select entity alias e
	• **FROM** Employee e – entity and alias
	• **WHERE** – filter conditions
	• **:dept, :minSalary** – parameters
	• **ORDER BY** – sort results

JPQL Features

- Object-oriented: uses entities and fields
- Automatic joins on mapped relationships
- Supports parameters (named or positional)
- Type-safe and validated at runtime
- Database independent

Common Clauses in JPQL

Clause	Description
SELECT	Select entities or fields
FROM	Specify entity and alias
WHERE	Filter conditions
JOIN / LEFT JOIN	Join related entities
ORDER BY	Sort results
GROUP BY	Group results
HAVING	Filter grouped results

Benefits

Database Independent
JPQL is portable across different databases.

Type Safe
Validated at compile time reduces runtime errors.

Easy Maintenance
Works with entities, easier to refactor and maintain.

Tip
Use JPQL for most read operations. Use native queries for database-specific features or complex SQL.

NATIVE POSTGRESQL QUERIES

nativeQuery = true drops down to real PostgreSQL SQL -- the escape hatch for anything JPQL can't express.

```
@Query(value = "SELECT * FROM customer c "
             + "WHERE c.status = :status AND c.created_at >= :fromDate",
       nativeQuery = true)
List<CustomerEntity> findActiveSince(@Param("status") String status,
                                     @Param("fromDate") Instant fromDate);
```

KEY TAKEAWAY Native queries use real table and column names and real PostgreSQL syntax -- reach for one only when JPQL genuinely can't express what's needed, such as a PostgreSQL-specific function.

TRY IT YOURSELF — EXERCISE 3: DESIGN THE REPOSITORY QUERIES

Reinforces: repository interfaces, JpaRepository, derived query methods, multiple criteria, JPQL, and native queries as you just covered.

STEPS (notes/lab39-repository.md)

Step	What To Do
1 — Derived Methods	Write two derived method names and the SQL each generates.
2 — Multiple Criteria	Write a method finding by status and name fragment.
3 — When JPQL	Say when a derived name stops being the right tool.
4 — When Native	Give one case that justifies dropping to native PostgreSQL SQL.

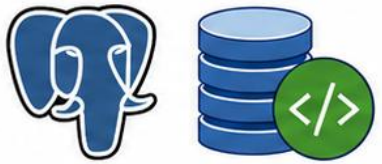
Repository methods

```
findByEmail(String email)                  -> where email = ?
findByStatusAndNameContainingIgnoreCase(..) -> where status = ? and lower(name) like ?
native: PostgreSQL-specific JSONB or window functions JPQL cannot express
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-repository-queries.md (workspace: examples/module-39-exercises).

Spring Data JPA

NATIVE POSTGRESQL QUERIES



Spring Data JPA allows you to execute native PostgreSQL SQL queries when JPQL is insufficient or when you need to use database-specific features.

When to Use Native Queries

- Database-specific functions (e.g., JSONB, ARRAY)
- Complex queries not easily expressed in JPQL
- Common Table Expressions (CTE)
- Window functions
- Performance tuning with hints
- Advanced joins or SQL constructs

Example Entity: Employee

```
@Entity
@Table(name = "employees")
public class Employee {
 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;
 private String firstName;
 private String lastName;
 private String department;
 private BigDecimal salary;
 private LocalDate hireDate;
 private Boolean active;
}
```

Repository Interface Using Native Queries

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

  @Query(value = "SELECT * FROM employees e WHERE e.department = :dept"
        AND e.salary > :minSalary ORDER BY e.salary DESC", nativeQuery = true)
  List<Employee> findByDepartmentAndMinSalaryNative(
      @Param("dept") String department,
      @Param("minSalary") BigDecimal minSalary);

  @Query(value = "SELECT e.id, e.first_name, e.last_name, d.name as department_name " +
       "FROM employees e JOIN departments d ON e.department = d.id " +
       "WHERE e.active = true", nativeQuery = true)
  List<Object[]> findActiveEmployeesWithDepartment();

  @Query(value = "SELECT * FROM employees e " +
       "WHERE e.hire_date BETWEEN :startDate AND :endDate", nativeQuery = true)
  List<Employee> findByHireDateBetweenNative(
      @Param("startDate") LocalDate startDate,
      @Param("endDate") LocalDate endDate);
}
```

Native Query Examples

Use Case	Example Query (PostgreSQL)
JSONB Filtering	`SELECT * FROM employees WHERE data ->> 'status' = 'ACTIVE'`
Array Contains	`SELECT * FROM employees WHERE skills && ARRAY['Java','SQL']`
CTE Example	`WITH high_earners AS (SELECT * FROM employees WHERE salary > 100000) SELECT * FROM high_earners WHERE department = 'IT'`
Window Function	`SELECT id, first_name, salary, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) rn FROM employees`

PostgreSQL Features You Can Use

JSONB Operators
Query JSON/JSONB columns using ->, ->>, @>, ?

ARRAY Support
Use ANY, ALL, &&, unnest() functions

Window Functions
ROW_NUMBER(), RANK(), SUM() OVER () etc.

CTE (WITH Clause)
Write recursive or non-recursive CTEs

Full-Text Search
to_tsvector, to_tsquery, @@ operators

Important Notes

- Set nativeQuery = true in @Query annotation.
- Returns can be entities, projections, or Object[].
- Column names must match the database.
- Use @Modifying for INSERT, UPDATE, DELETE.
- Consider SQL injection—use parameters.

Benefits

Leverage PostgreSQL Power
Use advanced PostgreSQL features for complex needs.

Better Performance
Write optimized SQL for high-performance queries.

Greater Flexibility
Handle complex reporting and analytical queries.

Tip
Use native queries judiciously—prefer JPQL for portability and maintainability when possible.

RELATIONSHIPS OVERVIEW

JPA maps every relationship cardinality from Module 37's schema design using a dedicated annotation and an explicit owning side.

Annotation	Cardinality	Owning Side
@OneToOne	1 <-> 1	Either side; @JoinColumn marks the owner.
@OneToMany / @ManyToOne	1 <-> N	The Many side, via @ManyToOne -- it holds the foreign key.
@ManyToMany	N <-> N	Either side; a join table is created automatically.

KEY TAKEAWAY The owning side always holds the foreign key -- this is the entity-mapping mirror of exactly which table actually carries the FOREIGN KEY column in Module 37's schema.

Spring Data JPA

RELATIONSHIPS OVERVIEW

Relationships define how entities (tables) are associated with each other.
They enforce data integrity and enable efficient navigation across the domain model.

Key Concepts

Primary Key (PK)
Uniquely identifies a record in a table.

Foreign Key (FK)
References the primary key in another table.

Relationship
Logical association between two entities.

Referential Integrity
Ensures valid relationships between related records.

JPA Relationship Annotations

`@OneToOne`	– One-to-One
`@OneToMany`	– One-to-Many
`@ManyToOne`	– Many-to-One
`@ManyToMany`	– Many-to-Many
`@JoinColumn`	– Defines Foreign Key
`@JoinTable`	– Defines Join Table

Types of Relationships

1. One-to-One (1:1)

Each record in A is related to one record in B and vice versa.

User			UserProfile	
id (PK)	username	1 ⟷ 1	id (PK, FK)	email
1	alice		1	alice@example.com
2	bob		2	bob@example.com

Example: User ↔ UserProfile
Use Case: Each user has one profile.

2. One-to-Many (1:N)

One record in A can be related to many records in B. Each record in B relates to one A.

Department			Employee		
id (PK)	name	1 ⟷ N	id (PK)	name	dept_id (FK)
10	HR		101	John	20
20	IT		102	Jane	20
			103	Mark	10

Example: Department → Employees
Use Case: A department has many employees.

3. Many-to-One (N:1)

Many records in A relate to one record in B. Each record in A has only one B.

Employee				Department	
id (PK)	name	dept_id (FK)	N ⟷ 1	id (PK)	name
101	John	20		10	HR
102	Jane	20		20	IT
103	Mark	10			

Example: Employees → Department
Use Case: Many employees belong to one department.

4. Many-to-Many (N:M)

Many records in A relate to many records in B.

Student		N ⟷	Student_Course		N ⟷	Course	
id (PK)	name		student_id (FK)	course_id (FK)		id (PK)	title
1	Alice		1	10		10	Spring Data JPA
2	Bob		1	20		20	PostgreSQL
			2	10			

Example: Students ↔ Courses
Use Case: Students enroll in multiple courses and courses have multiple students.

Best Practices

- Model relationships based on business requirements.
- Prefer LAZY fetching for collections to improve performance.
- Keep relationships bidirectional only when necessary.
- Use proper cascading and orphan removal carefully.

Benefits

- Enforces referential integrity.
- Simplifies data navigation.
- Enables object-oriented domain modeling.
- Improves maintainability and scalability.

Tip

Always understand the cardinality (1, N, M) before designing your entities and relationships.

MAPPING RELATIONSHIPS

Four worked mapping examples from the Northstar CRM entities -- one for each relationship annotation.

@OneToOne

CustomerEntity <-> CustomerProfileEntity, @JoinColumn(name = "customer_id") on the owning side, unique = true.

@OneToMany

CustomerEntity.accounts, mappedBy = "customer" -- the inverse, non-owning side; read-only in practice.

@ManyToOne

AccountEntity.customer, @JoinColumn(name = "customer_id") -- the owning side, holds the FK.

@ManyToMany

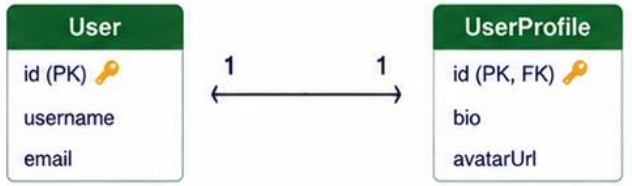
AccountEntity <-> TagEntity, @JoinTable generates account_tag automatically.

KEY TAKEAWAY @ManyToOne on AccountEntity.customer is the owning side and holds the real foreign key; @OneToMany on CustomerEntity.accounts, with mappedBy, is just a convenient read view of the same relationship from the other direction.

Spring Data JPA

ONE-TO-ONE MAPPING

A one-to-one relationship maps one record in an entity to exactly one record in another entity, and vice versa.



Concept Overview

Cardinality
Each User has one UserProfile.
Each UserProfile belongs to one User.

Foreign Key
UserProfile contains a foreign key that references User.

Use Cases
User and Profile, Employee and ID Card, Account and Account Settings, etc.

Key Benefit
Enforces data integrity and keeps related data tightly coupled.

Entity Example (JPA Annotations)

User Entity (Owning Side)

```
@Entity
@Table(name = "users")
public class User {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String username;
  private String email;

  @OneToOne(mappedBy = "user", cascade = CascadeType.ALL,
            orphanRemoval = true, fetch = FetchType.LAZY)
  private UserProfile profile;

  // getters and setters

}
```

UserProfile Entity (Inverse Side)

```
@Entity
@Table(name = "user_profiles")
public class UserProfile {

  @Id
  private Long id;

  private String bio;
  private String avatarUrl;

  @OneToOne
  @JoinColumn(name = "user_id", nullable = false,
              unique = true)
  private User user;

  // getters and setters

}
```

Mapping Details

Key Annotations

@OneToOne	Defines a one-to-one association.
@JoinColumn	Specifies the foreign key column.
mappedBy	Indicates the inverse (non-owning) side.
cascade	Propagates persistence operations.
fetch	Controls loading strategy (LAZY/EAGER).

Database Schema (Example)

users

Colume	
Column	Type
id (PK)	BIGINT
username	VARCHAR
email	VARCHAR

user_profiles

user_profiles	
Column	Type
id (PK)	BIGINT
bio	TEXT
avatar_url	VARCHAR
user_id (FK)	BIGINT (UNIQUE)

Example Usage

```
User user = new User();
user.setUsername("alice");
user.setEmail("alice@example.com");

UserProfile profile = new UserProfile();
profile.setBio("Software Engineer");
profile.setAvatarUrl("https://example.com/img.png");
profile.setUser(user);
user.setProfile(profile);

userRepository.save(user); // Cascades to save profile
```

Key Points

- One-to-one is implemented using @OneToOne.
- The owning side contains the foreign key (@JoinColumn).
- Use unique = true to enforce one-to-one at the DB level.
- Use cascade for automatic persist, merge, remove.
- Use orphanRemoval = true to delete the profile when it is removed from the user.

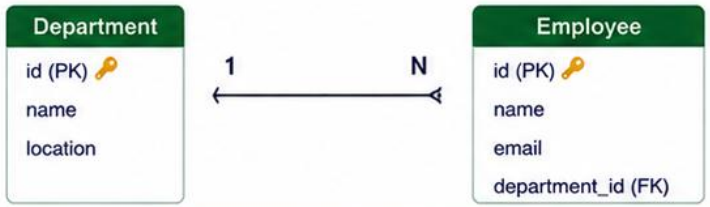
Best Practices

- Use LAZY fetching unless data is always needed together.
- Keep the relationship truly one-to-one (avoid misuse for one-to-many).
- Add database constraints (PK/FK, UNIQUE) for data integrity.
- Consider @MapsId for shared primary key one-to-one mappings when both entities share the same identifier.

Spring Data JPA

ONE-TO-MANY MAPPING

A one-to-many relationship maps one record in the parent entity to many records in the child entity.



Concept Overview

Cardinality
One Department has many Employees.
Each Employee belongs to one Department.

Foreign Key
The many side (Employee) contains a foreign key referencing the one side (Department).

Collection
Use a collection (List/Set) on the one side to hold related child entities.

Key Benefit
Models real-world hierarchies and supports cascading operations.

Entity Mapping (JPA Annotations)

Department Entity (One Side)

```
@Entity
@Table(name = "departments")
public class Department {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String name;
  private String location;

  @OneToMany(mappedBy = "department",
             cascade = CascadeType.ALL,
             orphanRemoval = true,
             fetch = FetchType.LAZY)
  private List<Employee> employees = new ArrayList<>();

  // getters and setters
}
```

Employee Entity (Many Side)

```
@Entity
@Table(name = "deployees")
public class Employee {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String name;
  private String email;

  @ManyToOne(fetch = FetchType.LAZY)
  @JoinColumn(name = "department_id", nullable = false)
  private Department department;


  // getters and setters
}
```

Database Schema (Example)

departments

Column	Type	Constraints
id	BIGINT	PRIMARY KEY
name	VARCHAR	NOT NULL
location	VARCHAR	NULL

employees

Column	Type	Constraints
id	BIGINT	PRIMARY KEY
name	VARCHAR	NOT NULL
email	VARCHAR	NOT NULL
department_id	BIGINT	FOREIGN KEY → departments(id)

Example Usage

```
Department dept = new Department();
dept.setName("Engineering");
dept.setLocation("Bengaluru");

Employee emp1 = new Employee();
emp1.setName("Alice");
emp1.setEmail("alice@example.com");
emp1.setDepartment(dept);

Employee emp2 = new Employee();
emp2.setName("Bob");
emp2.setEmail("bob@example.com");
emp2.setDepartment(dept);

// Add children to parent
dept.getEmployees().add(emp1);
dept.getEmployees().add(emp2);


departmentRepository.save(dept); // Cascades to save employees
```

Key Points

- `@OneToMany` is on the one side (Department).
- `@ManyToOne` is on the many side (Employee).
- `mappedBy = "department"` avoids a join table.
- `@JoinColumn` defines the foreign key column.
- Use `CascadeType.ALL` to propagate persist, merge, remove operations.
- Use `FetchType.LAZY` for better performance.

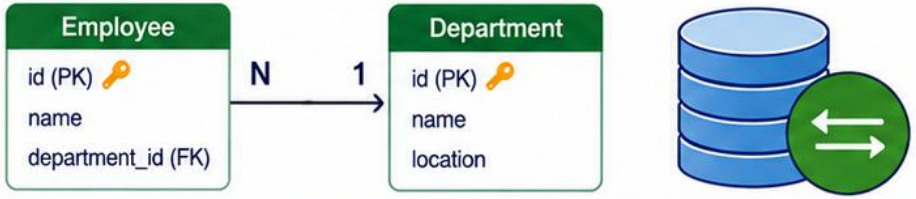
Best Practices

- Keep the relationship unidirectional from many-to-one side only unless you need navigation both ways.
- Prefer LAZY loading for collections.
- Use `orphanRemoval = true` to delete children when removed from the collection.
- Avoid business logic in entities, keep entities as data carriers.
- Add database constraints for referential integrity.

Spring Data JPA

MANY-TO-ONE MAPPING

A many-to-one relationship maps many records in a child entity to a single record in a parent entity.



Concept Overview

Cardinality
Many employees can belong to one department.

Foreign Key
Employee table contains a foreign key that references Department.

Collection
Parent (Department) does not need a collection in a many-to-one mapping.

Key Benefit
Avoids data duplication and maintains referential integrity.

Entity Mapping (JPA Annotations)

Many-Side Entity (Employee)

```
@Entity
@Table(name = "employees")
public class Employee {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String name;
  private String email;

  @ManyToOne(fetch = FetchType.LAZY)
  @JoinColumn(name = "department_id", nullable = false)
  private Department department;

  // getters and setters
}
```

One-Side Entity (Department)

```
@Entity
@Table(name = "departments")
public class Department {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String name;
  private String location;

  // No collection needed for many-to-one

  // getters and setters
}
```

Database Schema (Example)

departments

Column	Type	Constraints
id	BIGINT	PRIMARY KEY
name	VARCHAR	NOT NULL
location	VARCHAR	NULL

employees

Column	Type	Constraints
id	BIGINT	PRIMARY KEY
name	VARCHAR	NOT NULL
email	VARCHAR	NOT NULL
department_id	BIGINT	FOREIGN KEY → departments(id)

Example Usage

```
Department dept = departmentRepository.findById(1L).get();

Employee emp = new Employee();
emp.setName("Alice");
emp.setEmail("alice@example.com");
emp.setDepartment(dept);

employeeRepository.save(emp);
// The employee is linked to one department
```

Key Points

- **@ManyToOne** is used on the child entity.
- **@JoinColumn** specifies the foreign key column.
- The child contains the foreign key.
- The parent does not need a collection.
- Uses **FetchType.LAZY** for better performance.
- Maintains referential integrity.

Best Practices

- Use **@ManyToOne** for many-to-one relationships.
- Always specify @JoinColumn.
- Prefer **FetchType.LAZY** to avoid unnecessary joins.
- Keep foreign keys non-nullable when required.
- Ensure proper indexing on foreign key columns.
- Follow consistent naming conventions.

MODULE 39

Spring Data JPA and PostgreSQL Integration

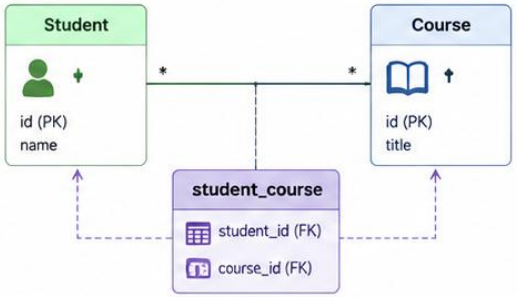
Many-to-Many Mapping

Modeling a many-to-many relationship using JPA with a join table.

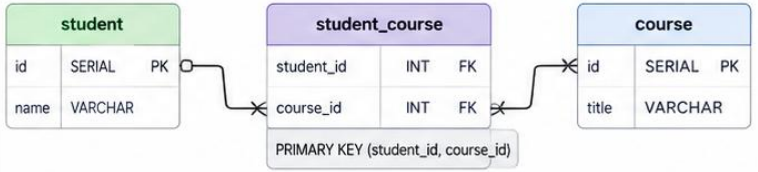
CONCEPT

- A many-to-many relationship exists when multiple records of one entity are associated with multiple records of another entity.
- JPA implements many-to-many relationships using a join table that contains foreign keys to both entities.
- The join table breaks the relationship into two many-to-one relationships.

EXAMPLE DOMAIN



DATABASE REPRESENTATION



KEY POINTS

- Use @ManyToMany on both sides.
- Define a join table with @JoinTable.
- Use mappedBy on one side to avoid duplicate join tables.
- CascadeType carefully – avoid CascadeType.REMOVE.
- FetchType.LAZY is recommended.

STUDENT ENTITY

```
@Entity
@Table(name = "student")
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}
```

COURSE ENTITY

```
@Entity
@Table(name = "course")
public class Course {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToMany(mappedBy = "courses", fetch = FetchType.LAZY)
    private Set<Student> students = new HashSet<>();
}
```

BEST PRACTICE: Use Set instead of List to prevent duplicate associations. Keep the relationship unidirectional unless you need navigation from both sides. Prefer LAZY fetching and handle performance with DTOs or projections.

Spring Data JPA → Hibernate (JPA) → PostgreSQL

FETCH STRATEGIES

Fetch strategy controls WHEN JPA loads a related entity or collection relative to when the owning entity itself loads.

Annotation	Default Fetch Type
@ManyToOne	EAGER -- loaded immediately, alongside the owning entity.
@OneToOne	EAGER -- loaded immediately, alongside the owning entity.
@OneToMany	LAZY -- loaded only on first access to the collection.
@ManyToMany	LAZY -- loaded only on first access to the collection.

KEY TAKEAWAY The defaults are easy to get backwards from intuition -- singular associations default EAGER, collections default LAZY -- and this course overrides @ManyToOne to LAZY deliberately.

MODULE 39

Spring Data JPA and PostgreSQL Integration

FETCH STRATEGIES

Controlling how related entities are loaded in JPA.

CONCEPT

Fetch strategies define when and how related entities are loaded from the database.

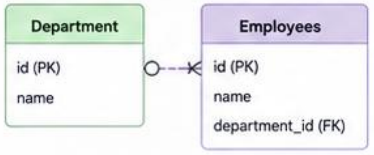
- **LAXY** (default) loads the association on demand.
- **EAGER** loads the association immediately with the parent entity.

Choosing the right fetch strategy improves performance and avoids unnecessary queries.

FETCH STRATEGY COMPARISON

Strategy	When Loaded	How It Works	Pros	Cons	Use When
LAZY (default)	On demand (when accessed)	Initial query loads only the parent entity. Related entity is loaded when accessed.	✓ Better performance for large associations ✓ Avoids unnecessary data loading	✗ May cause N+1 queries ✗ LazyInitializationException outside transaction	Association is large or not always needed
EAGER	Immediately (with parent)	Related entity is loaded together with the parent entity in the initial query.	✓ No additional queries when accessing ✓ Simple to use	✗ Can load unnecessary data ✗ Performance impact on large associations	Association is small and always needed

LAZY FETCH (Default)




```
@OneToMany(fetch = FetchType.LAZY)
```

1. Initial Query (Parent Only)

```
SELECT * FROM department
```

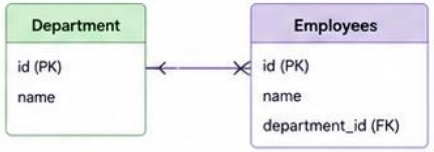
2. Access Collection

```
department.getEmployees()
```

3. Additional Query

```
SELECT * FROM employees
WHERE department_id = ?
```

EAGER FETCH




```
@OneToMany(fetch = FetchType.EAGER)
```

1. Single Query (Join)

```
SELECT d.*, e.*
FROM department d
LEFT JOIN employees e
ON d.id = e.department_id
```

✓ Both Department and its Employees are loaded in the same query.

KEY POINTS

- **LAZY** is the default for @OneToMany and @ManyToMany.
- **EAGER** is the default for @ManyToOne and @OneToOne.
- **LAZY** associations require an active persistence context.
- Avoid **EAGER** on collections to prevent performance issues.
- Use JOIN FETCH in queries to load associations efficiently.

COMMON USE CASES

LAZY	Large collections (e.g., Department → Employees) Optional data that is not always needed
EAGER	Small reference data (e.g., Country → Currency) Data that is always required with the parent
JOIN FETCH	Read-heavy use cases Reporting and dashboard queries

BEST PRACTICES

- Prefer LAZY and load explicitly when needed.
- Use JOIN FETCH or EntityGraph to avoid N+1 queries.
- Keep transactions short and service-layer boundaries clean.
- Test queries with realistic data and analyze execution plans.
- Monitor query count and response time regularly.

TIP: Use LAZY by default for collections. Use **EAGER** only when the associated data is small and always required.

LAZY VS EAGER LOADING

LAZY loads on demand; EAGER loads immediately -- the wrong choice either wastes queries or risks a LazyInitializationException.

Aspect	LAZY	EAGER
When fetched	On first access to the field/collection.	Immediately, with the owning entity.
Query shape	May trigger an additional query later.	Uses a JOIN -- one query.
Risk	LazyInitializationException if accessed outside a transaction.	Can silently over-fetch, or contribute to N+1.

KEY TAKEAWAY Lab 39 deliberately overrides @ManyToOne's EAGER default to LAZY on AccountEntity.customer -- loading accounts should not automatically load every account's full parent customer too.

MODULE 39

Spring Data JPA and PostgreSQL Integration

LAZY VS EAGER LOADING

Understanding how JPA loads related entities.

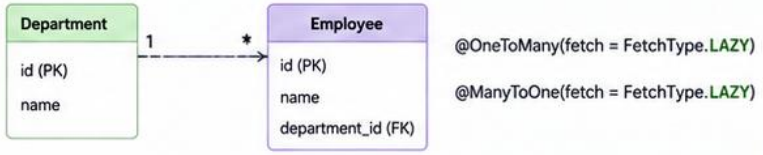
CONCEPT

- JPA associations can be loaded in two ways: LAZY or EAGER.
- LAZY loads the association only when it is accessed.
- EAGER loads the association immediately when the parent entity is fetched.
- Choosing the right strategy impacts performance and the number of SQL queries.

KEY TAKEAWAYS

- LAZY is the default for @OneToMany and @ManyToMany.
- EAGER is the default for @ManyToOne and @OneToOne.
- Use LAZY for large collections.
- Use EAGER for small, always-needed references.
- Avoid EAGER on collections to prevent N+1 query problems.

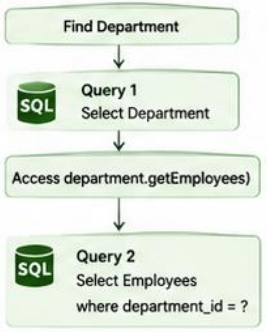
LAZY LOADING (Default for Collections)



How it works

1. When you load `Department`, JPA executes only 1 query to fetch the `Department`.
2. `Employees` are not loaded until you access `department.getEmployees()`.
3. Then a separate query is executed to load `Employees`.

Query Flow



Pros

- Reduced initial load time.
- Better performance for large collections.
- Loads data only when needed.

Cons

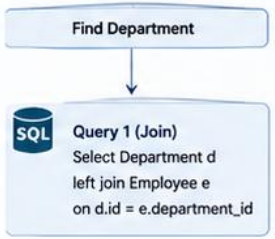
- Additional query when accessed.
- Can cause LazyInitializationException outside transaction.

EAGER LOADING

How it works

1. When you load Department, JPA executes 1 query and joins Employees.
2. Both `Department` and `Employees` are loaded immediately.

Query Flow



Pros

- Data is available immediately.
- No additional query when accessing the association.

Cons

- Loads unnecessary data.
- Can impact performance for large collections.
- May cause N+1 queries with nested associations.

BEST PRACTICES

- Use LAZY for collections (@OneToMany, @ManyToMany) by default.
- Use EAGER only for small, frequently accessed reference data.
- Use JOIN FETCH or EntityGraph to fetch associations in queries when needed.
- Keep transactions open during access to avoid LazyInitializationException.
- Always analyze queries with real data using **EXPLAIN ANALYZE**.

RECOMMENDATION

Prefer LAZY loading as the default strategy. Explicitly fetch associations in queries only when required by the use case.

N+1 QUERY PROBLEM

The exact N+1 pattern from Module 38, now shown as something JPA can generate automatically without an explicit loop in sight.

```
List<CustomerEntity> customers = customerRepository.findAll();  // 1 query
for (CustomerEntity c : customers) {
    c.getAccounts().size();   // +1 query PER customer, if accounts is LAZY
}

// Fix: fetch both in one query
@Query("SELECT c FROM CustomerEntity c JOIN FETCH c.accounts")
List<CustomerEntity> findAllWithAccounts();
```

KEY TAKEAWAY This is Module 38's N+1 problem returning in a JPA-specific form -- the loop looks completely innocent because a LAZY collection access hides a real query behind a simple method call.

MODULE 39

Spring Data JPA and PostgreSQL Integration

N+1 QUERY PROBLEM

Understanding the issue and how to prevent unnecessary database round-trips.

CONCEPT

- The N+1 query problem occurs when we load a list of parent entities (1 query) and then access a lazy-loaded association for each parent (N additional queries).
- This leads to N+1 total queries and poor performance.

IMPACT

- High number of database round-trips
- Increased latency
- Poor application performance
- Does not scale with large datasets

SOLUTIONS

- Use JOIN FETCH in JPQL
- Use EntityGraph
- Use batch fetching (@BatchSize)
- Use DTO projections
- Avoid EAGER by default

THE PROBLEM: N+1 QUERIES (Using LAZY Loading)

Java Application

```
List<Department> departments =
  departmentRepository.findAll();
for (Department d : departments) {
   d.getEmployees().size(); // Triggers query
 }
```

Database

N additional queries

1. `SELECT * FROM department;` → 1 query
2. `SELECT * FROM employee WHERE department_id = 1;` → 1 query
3. `SELECT * FROM employee WHERE department_id = 2;` → 1 query

⋮

N+1. `SELECT * FROM employee WHERE department_id = N;` → 1 query

Total Queries = 1 (for departments) + N (for employees) = N+1 queries

EXAMPLE

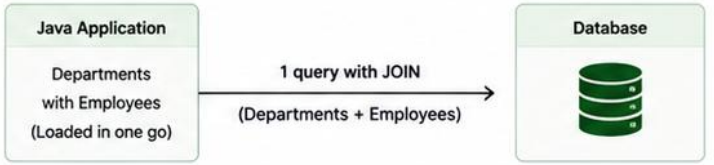
If there are 100 departments, total queries executed = 1 + 100 = 101 queries

This severely impacts performance.

THE SOLUTION: FETCH IN ONE QUERY

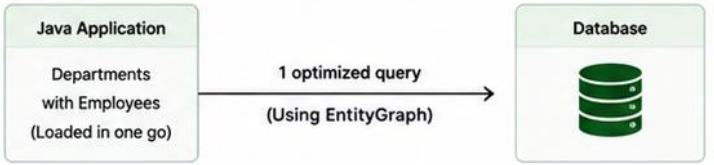
1. JOIN FETCH (JPQL)

```
SELECT d FROM Department d
JOIN FETCH d.employees
```



2. ENTITY GRAPH

```
@EntityGraph(attributePaths = "employees")
List<Department> findAll();
```



BEST PRACTICES

- Always analyze query count during development (e.g., using Hibernate Statistics).
- Use JOIN FETCH or EntityGraph for collections when you know you need the data.
- Use batch fetching when JOIN FETCH is not possible.
- Avoid EAGER on collections; use LAZY with proper fetching strategies.
- Test with realistic data volumes to detect performance issues early.

KEY TAKEAWAY

N+1 happens with LAZY associations.

Fetch smartly, not eagerly!

Optimize queries, optimize performance.

PAGINATION WITH PAGEABLE

Pageable defines a page request; Page<T> returns the data plus total-count metadata -- Spring Data JPA's built-in answer to Module 38's LIMIT/OFFSET.

```
Page<CustomerEntity> findByStatus(String status, Pageable pageable);

int safeSize = Math.min(Math.max(size, 1), 100);
Pageable page = PageRequest.of(number, safeSize, Sort.by("fullName"));
Page<CustomerEntity> result = customerRepository.findByStatus("ACTIVE", page);
```

KEY TAKEAWAY Never let a client request an unbounded page size -- Lab 39 caps size at 100 and always adds a deterministic tie-breaker to the sort, exactly like Module 38's pagination guidance.

MODULE 39

Spring Data JPA and PostgreSQL Integration

PAGINATION WITH PAGEABLE

Handling large result sets efficiently using Spring Data JPA Pageable.

CONCEPT

- Pagination splits large result sets into smaller pages.
- Spring Data JPA provides *Pageable* and *Page<T>* to simplify pagination.
- Supports page number, page size, sorting, and total count.
- Improves performance and reduces memory usage.

KEY BENEFITS

- Efficiently handles large datasets
- Reduces database load and memory usage
- Provides total count and total pages
- Supports sorting with pagination
- Easy integration with REST APIs

1. REPOSITORY METHOD

```
@Repository
public interface EmployeeRepository
        extends JpaRepository<Employee, Long> {

    // Spring Data JPA automatically implements this method
    Page<Employee> findAll(Pageable pageable);
}
```

2. SERVICE LAYER EXAMPLE

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    public Page<Employee> getAllEmployees(int page, int size, String sortBy, String dir) {
        Sort sort = dir.equalsIgnoreCase("desc") ?
            Sort.by(sortBy).descending() : Sort.by(sortBy).ascending();
        Pageable pageable = PageRequest.of(page, size, sort);
        return repository.findAll(pageable);
    }
}
```

3. CONTROLLER EXAMPLE (REST API)

```
@GetMapping("/api/employees")
public Page<Employee> getEmployees(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "id") String sortBy,
    @RequestParam(defaultValue = "asc") String dir) {
    return employeeService.getAllEmployees(page, size,
}   sortBy, dir);
```

COMMON REQUEST URI

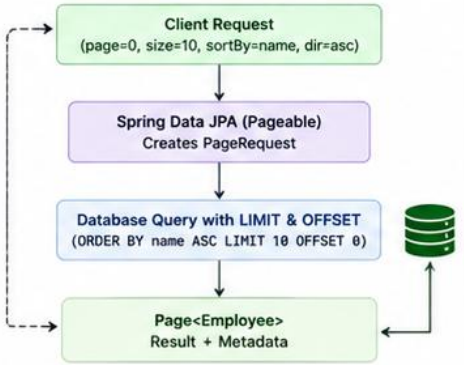
```
GET /api/employees?page=0&size=10&sortBy=name&dir=asc
```

- page: Page number (0-based index)
- size: Number of records per page
- sortBy: Field to sort by
- dir: Sort direction (asc | desc)

4. PAGE<T> RESPONSE STRUCTURE

```
{
  "content": [
     { "id": 1, "name": "Alice" },      → List of records in the current page
     { "id": 2, "name": "Bob" }
  ],
  "pageable": {                         → Paging and sorting information
     "pageNumber": 0,
     "pageSize": 10,
     "sort": { "sorted": true }
  },
  "totalElements": 105,                 → Total number of matching records
  "totalPages": 11,                     → Total number of pages
  "last": false,                        → Is this the last page?
  "numberOfElements": 10,               → Number of records in this page
  "size": 10,                           → Page size
  "number": 0,                          → Current page number (0-based)
  "first": true,                        → Is this the first page?
  "empty": false                        → Is the page empty?
}
```

5. HOW PAGINATION WORKS



6. PAGINATION EXAMPLE

Page (Number)	Size	Sort	SQL Generated (PostgreSQL)	Records Returned
0	10	name ASC	`SELECT * FROM employee ORDER BY name ASC LIMIT 10 OFFSET 0;`	10
1	10	name ASC	`SELECT * FROM employee ORDER BY name ASC LIMIT 10 OFFSET 10;`	10
2	10	name ASC	`SELECT * FROM employee ORDER BY name ASC LIMIT 10 OFFSET 20;`	10
...	...	...	....	...

BEST PRACTICES

- Always use pagination for large result sets.
- Define a default sort order for consistency.
- Use appropriate page size (e.g., 10, 20, 50).
- Avoid large page sizes to prevent memory issues.
- Expose pagination parameters via API (query params).
- Monitor slow queries and optimize indexes for sorting fields.

QUICK REFERENCE

```
Pageable pageable = PageRequest.of(page, size, Sort.by("name").ascending());
Page<Employee> page = repository.findAll(pageable);
List<Employee> content = page.getContent();
long total = page.getTotalElements();
int totalPages = page.getTotalPages();
```

SORTING

Sort defines field order for a query -- combine it with Pageable for deterministic, paginated results.

```
Sort.by("fullName").ascending();
Sort.by("status").ascending().and(Sort.by("fullName").descending());

Pageable pageable = PageRequest.of(0, 20,
    Sort.by("fullName").ascending().and(Sort.by("id").ascending()));
```

KEY TAKEAWAY Always add a unique tie-breaker field, like id, to the end of a sort used with pagination -- without one, rows with equal sort values can shift between pages unpredictably.

MODULE 39

Spring Data JPA and PostgreSQL Integration

SORTING

Ordering query results using Spring Data JPA.

CONCEPT

- Sorting arranges results based on one or more properties.
- Spring Data JPA provides *Sort* and *Pageable* to apply sorting.
- Supports ascending (ASC) and descending (DESC) order.
- Multiple fields can be sorted to ensure consistent results.

1. SORT INTERFACE

The *Sort* interface defines sorting instructions.

```
Sort sort = Sort.by(Sort.Direction.ASC, "name");
Sort sortDesc = Sort.by(Sort.Direction.DESC, "createdAt");
Sort multiSort = Sort.by(
    Sort.Order.asc("department"),
    Sort.Order.desc("name")
);
```

2. REPOSITORY EXAMPLES

Using *Sort* in repository methods.

```
List<Employee> findAll(Sort sort);

Page<Employee> findAll(Pageable pageable);

List<Employee> findByDepartmentId(Long departmentId,
                Sort sort);
```

3. CONTROLLER EXAMPLE (REST API)

```
@GetMapping("/api/employees")
public Page<Employee> getEmployees(
      @RequestParam(defaultValue = "0") int page,
      @RequestParam(defaultValue = "10") int size,
      @RequestParam(defaultValue = "name") String sortBy,
      @RequestParam(defaultValue = "asc") String direction) {

   Sort sort = direction.equalsIgnoreCase("desc") ?
       Sort.by(sortBy).descending() : Sort.by(sortBy).ascending();

   Pageable pageable = PageRequest.of(page, size, sort);
   return employeeService.getEmployees(pageable);
}
```

KEY POINTS

- *Sort* results by any entity property.
- Use *Sort* for simple sorting needs.
- Use *Pageable* for pagination + sorting.
- Multiple fields ensure deterministic ordering.
- Default sorting prevents inconsistent results between pages.

4. SORTING EXAMPLES

A. Single Field Sort

```
SELECT * FROM employee
ORDER BY name ASC;
```

ID	Name	Department
1	Alice	HR
2	Bob	IT
3	Charlie	Finance
4	David	IT

A → Z

B. Single Field Sort (Descending)

```
SELECT * FROM employee
ORDER BY created_at DESC;
```

ID	Name	Created At
4	David	2024-05-10
2	Bob	2024-05-08
3	Charlie	2024-05-05
1	Alice	2024-05-01

Z → A

C. Multiple Field Sort

```
SELECT * FROM employee
ORDER BY department ASC,name ASC;
```

ID	Name	Department
1	Alice	Finance
2	Bob	HR
3	Charlie	HR
1	David	IT

Department (A -> Z) ↕ Then Name (A -> Z)

5. COMMON SORT FIELDS

- String fields (e.g., name)
- Numeric fields (e.g., salary)
- Date/Time fields (e.g., createdAt)
- Enum fields (e.g., status)
- Nested properties (e.g., department.name)

6. EXAMPLE REQUEST URIS

```
GET /api/employees?sortBy=name&dir=asc
GET /api/employees?sortBy=createdAt&dir=desc
GET /api/employees?sortBy=department,name&dir=asc
GET /api/employees?page=1&size=10&sortBy=salary&dir=desc
```

BEST PRACTICES

- Always provide a default sort field and direction.
- Use indexed columns for better sorting performance.
- Avoid sorting on large text/blob fields.
- Use multi-field sorting to ensure consistent order across pages.
- Monitor query execution plans for sort operations on large datasets.

QUICK REFERENCE

```
Sort sort = Sort.by("name").ascending();
Sort sortDesc = Sort.by("createdAt").descending();
Sort multiSort = Sort.by(Sort.Order.asc("department"), Sort.Order.desc("name"));
Pageable pageable = PageRequest.of(page, size, sort);
List<Employee> list = repository.findAll(sort);
Page<Employee> pageData = repository.findAll(pageable);
```

TRY IT YOURSELF — EXERCISE 4: CHOOSE FETCH STRATEGY AND KILL N+1

Reinforces: relationship mapping, fetch strategies, lazy versus eager, the N+1 problem, Pageable, and sorting as you just covered.

STEPS (notes/lab39-fetch-strategy.md)

Step	What To Do
1 — Defaults	State the default fetch type for to-one and to-many relationships.
2 — N+1 Cause	Say what produces N+1 when listing customers with their accounts.
3 — The Fix	Name the fix, and say why switching to EAGER is not it.
4 — Pageable	Show the repository signature that returns one page, sorted.

Fetching

```
@ManyToOne default EAGER   @OneToMany default LAZY
N+1: 1 list query + 1 per customer for accounts
fix: join fetch / @EntityGraph -- not EAGER, which N+1s everywhere
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-fetch-and-nplus1.md (workspace: examples/module-39-exercises).

TRANSACTION INTEGRATION

Spring integrates Spring Data JPA transactions with Hibernate and PostgreSQL to guarantee the same ACID properties Module 37 taught at the database level.

ACID Property	What @Transactional Guarantees
Atomicity	Every repository call inside the method succeeds together, or none commit.
Consistency	The database moves from one valid state to another -- constraints hold.
Isolation	Concurrent transactions don't see each other's uncommitted changes.
Durability	Once the method returns successfully, the change survives a crash.

KEY TAKEAWAY @Transactional at the service layer is Spring's mechanism for delivering exactly the ACID guarantees Module 37 defined at the raw SQL BEGIN/COMMIT level -- now automatic, around a whole Java method.

MODULE 39

Spring Data JPA and PostgreSQL Integration

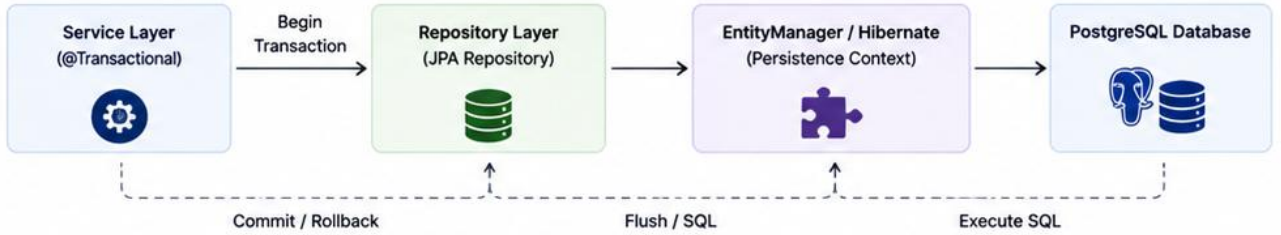
TRANSACTION INTEGRATION

Managing database transactions in Spring Data JPA for consistency and reliability.

CONCEPT

- A transaction is a sequence of operations performed as a single unit of work.
- Spring Data JPA integrates with Spring's transaction management to ensure ACID properties.
- If any operation fails, the entire transaction is rolled back.
- Use @Transactional to declare transactional boundaries.

HOW TRANSACTION INTEGRATION WORKS



ACID IN ACTION

- **A Atomicity** — All operations succeed or none are applied.
- **C Consistency** — Database remains in a valid state.
- **I Isolation** — Transactions do not interfere with each other.
- **D Durability** — Committed changes are permanent.

KEY BENEFITS

- Ensures data consistency and integrity
- Automatic commit on success
- Automatic rollback on failure
- Reduces boilerplate transaction code
- Supports read-only optimization
- Works seamlessly with repositories and services

EXAMPLE: SERVICE LAYER TRANSACTION

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;
    private final DepartmentRepository departmentRepository;

    @Transactional
    public void transferEmployee(Long employeeId, Long newDepartmentId) {
        Employee employee = employeeRepository.findById(employeeId)
            .orElseThrow(() -> new RuntimeException("Employee not found"));

        Department newDept = departmentRepository.findById(newDepartmentId)
            .orElseThrow(() -> new RuntimeException("Department not found"));

        employee.setDepartment(newDept);
        // Both operations executed within the same transaction
        employeeRepository.save(employee);
        departmentRepository.save(newDept);
        // If any exception occurs, the transaction is rolled back
    }
}
```

TRANSACTION PROPERTIES

Attribute	Description	Example
propagation	Behavior when a transactional method is called from another transactional method.	`REQUIRED` (default), `REQUIRES_NEW`, `SUPPORTS`, `MANDATORY`, `NEVER`, `NESTED`
Isolation	Isolation level of the transaction.	`DEFAULT`, `READ_UNCOMMITTED`, `READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`
readOnly	Hint for optimization when no data modification is performed.	`@Transactional(readOnly = true)`
timeout	Maximum time (in seconds) for the transaction.	`@Transactional(timeout = 30)`
rollbackFor	Exceptions that should trigger a rollback.	`@Transactional(rollbackFor = Exception.class)`

DEFAULT BEHAVIOR

`@Transactional`

- Propagation: REQUIRED
- Isolation: DEFAULT (database default)
- ReadOnly: false
- Rollback on: Unchecked exceptions
- (RuntimeException, Error)

FLOW ON SUCCESS vs FAILURE

SUCCESS FLOW	FAILURE FLOW
1. Begin Transaction	1. Begin Transaction
2. Execute Operations	2. Exception Occurs
3. Commit	3. Rollback
4. Changes Persisted	4. No Changes Persisted

BEST PRACTICES

- Keep transactions short and focused.
- Define transaction boundaries in the service layer.
- Use readOnly = true for read operations.
- Handle exceptions properly to avoid unexpected rollbacks.
- Avoid long-running transactions.
- Understand and use the right propagation level.

QUICK REFERENCE

```
@Transactional
public void doWork() { ... }
@Transactional(readOnly = true)
public List<Employee> findAllEmployees() { ... }
```

@TRANSACTIONAL WITH JPA

@Transactional draws a transaction boundary around a service method -- place it at the service layer, never on repositories.

```
@Service
public class CustomerService {
    @Transactional
    public CustomerResponse create(CreateCustomerRequest req) {
        String email = normalize(req.email());
        if (repo.existsByNormalizedEmail(email)) {
            throw new DuplicateCustomerException();
        }
        var saved = repo.save(toEntity(req));
        return toResponse(saved);
    }
}
```

KEY TAKEAWAY Spring rolls back automatically on any RuntimeException, including DuplicateCustomerException here -- but never on a checked exception unless rollbackFor says otherwise.

MODULE 39

Spring Data JPA and PostgreSQL Integration

@TRANSACTIONAL WITH JPA

Managing transactions in Spring Data JPA for data consistency and reliability.

CONCEPT

- @Transactional defines transactional boundaries for methods or classes.
- Ensures that a group of database operations are executed atomically.
- If any operation within the transaction fails, the entire transaction is rolled back.
- In Spring Data JPA, transactions are handled by Spring's transaction management.

KEY BENEFITS

- Ensures ACID compliance
- Automatic commit on success
- Automatic rollback on failure
- Reduces boilerplate transaction code
- Consistent data integrity
- Easy declarative transaction management

EXAMPLE: @TRANSACTIONAL USAGE

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;
    private final DepartmentRepository departmentRepository;

    public EmployeeService(EmployeeRepository employeeRepository,
            DepartmentRepository departmentRepository) {
        this.employeeRepository = employeeRepository;
        this.departmentRepository = departmentRepository;
    }

    @Transactional
    public void transferEmployee(Long employeeId, Long newDepartmentId) {
        Employee employee = employeeRepository.findById(employeeId)
          .orElseThrow(() -> new RuntimeException("Employee not found"));

        Department newDept = departmentRepository.findById(newDepartmentId)
          .orElseThrow(() -> new RuntimeException("Department not found"));

        employee.setDepartment(newDept);
        employeeRepository.save(employee);
        // If any exception occurs, the transaction is rolled back
    }
}
```

What happens?

1. **Begin Transaction**
   Spring opens a transaction before method execution.
2. **Execute Operations**
   All JPA operations are executed within the transaction.
3. **Commit**
   If all operations succeed, transaction is committed.
4. **Rollback**
   If any operation fails, entire transaction is rolled back.

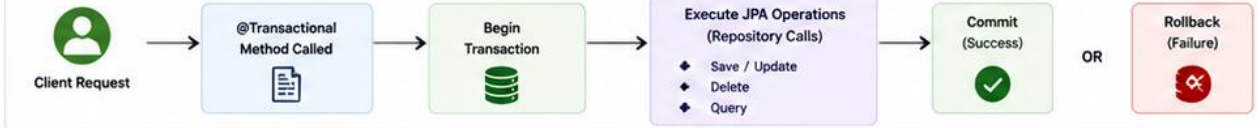
TRANSACTION PROPAGATION

Propagation	Description	Use Case
REQUIRED (default)	Join existing transaction or create a new one.	Most common scenario
REQUIRES_NEW	Suspends existing transaction and creates a new one.	Audit logging, notifications
SUPPORTS	Join existing transaction if present, otherwise execute non-transactionally.	Read-only operations
NOT_SUPPORTED	Execute non-transactionally, suspends existing transaction.	Performance-critical reads
MANDATORY	Must run within an existing transaction, otherwise throws exception.	Enforce transactional context
NEVER	Must run outside a transaction, otherwise throws exception.	Non-transactional operations
NESTED	Create nested transaction with a savepoint.	Partial rollback scenarios

@TRANSACTIONAL ATTRIBUTES

Attribute	Description	Example
propagation	Defines how transactions propagate.	`@Transactional(propagation = Propagation.REQUIRED)`
isolation	Defines the isolation level.	`@Transactional(isolation = Isolation.READ_COMMITTED)`
readOnly	Hint that the transaction is read-only.	`@Transactional(readOnly = true)`
timeout	Maximum time (in seconds) for the transaction.	`@Transactional(timeout = 30)`
rollbackFor	Specify exceptions to trigger rollback.	`@Transactional(rollbackFor = Exception.class)`
noRollbackFor	Specify exceptions that should not trigger rollback.	`@Transactional(noRollbackFor = CustomException.class)`

TRANSACTION FLOW



BEST PRACTICES

- Keep transactions short and focused.
- Do not perform remote calls or long operations in transactions.
- Use `readOnly = true` for read-only operations.
- Handle exceptions properly to trigger rollback.
- Avoid nested transactions unless necessary.
- Use appropriate propagation level for each use case.

QUICK REFERENCE

```
@Transactional
@Transactional(readOnly = true)
@Transactional(timeout = 30)
@Transactional(propagation = Propagation.REQUIRES_NEW)
@Transactional(rollbackFor = Exception.class)
@Transactional(isolation = Isolation.READ_COMMITTED)
```

PERSISTENCE EXCEPTIONS

JPA and Spring wrap low-level database errors into a consistent hierarchy of unchecked exceptions.

Exception	Typical Cause
EntityNotFoundException	A referenced entity ID doesn't exist -- e.g. an invalid lazy reference.
DataIntegrityViolationException	A database constraint was violated -- NOT NULL, UNIQUE, CHECK, or a foreign key.
OptimisticLockingFailureException	A concurrent update conflicted with an @Version-checked entity.
JpaSystemException	A general, less specific JPA/Hibernate-level failure.

KEY TAKEAWAY Every one of these is an unchecked RuntimeException, so a @Transactional method automatically rolls back when one is thrown -- exactly the rollback rule from the previous slide, applied.

Persistence Exceptions

Persistence exceptions occur when Hibernate/JPA operations fail due to database constraints, invalid data, or transactional issues.

Exception Translation in Spring

Spring translates low-level JPA and JDBC exceptions into the unified **DataAccessException** hierarchy, making exception handling consistent and portable.

Common Persislation Exceptions

Exception	When It Occurs
DataIntegrityViolationException	Violates constraints (unique, foreign key, not null, check)
OptimisticLockingFailureException	Version mismatch during concurrent update
EntityNotFoundException	Requested entity not found in the database
InvalidDataAccessResourceUsageException	Invalid query, bad SQL, or misuse of JPA API
TransientDataAccessException	Temporary database connectivity issues

Exception Handling Best Practices

- Catch specific exceptions and handle appropriately.
- Translate exceptions into meaningful business messages.
- Avoid exposing low-level database details to clients.
- Log full exception details for troubleshooting.
- Use `@Transactional` to manage rollbacks automatically.

Example: Unique Constraint Violation

```
try {
    userRepository.save(user);
} catch (DataIntegrityViolationException ex) {
    // Duplicate email or unique constraint violation
    throw new BusinessException("Email already exists.");
}
```

Key Takeaway
Proper handling of persistence exceptions ensures data integrity, better user experience, and easier debugging in enterprise applications.

Robust exception handling is essential for reliable and maintainable data-driven applications.

POSTGRESQL CONSTRAINT VIOLATIONS

The exact PostgreSQL SQLSTATE codes from Module 37's constraints, now surfacing as Spring exceptions a service can catch and translate.

PostgreSQL Constraint	SQLSTATE	Spring Exception
UNIQUE violation	23505	DataIntegrityViolationException
NOT NULL violation	23502	DataIntegrityViolationException
FOREIGN KEY violation	23503	DataIntegrityViolationException
CHECK violation	23514	DataIntegrityViolationException

KEY TAKEAWAY All four Module 37 constraint types collapse into the same DataIntegrityViolationException in Java -- a service layer that needs to distinguish them inspects the wrapped SQLSTATE code.

PostgreSQL Constraint Violations

Constraint violations occur when data operations break defined rules in the database, ensuring data integrity and consistency.

Common PostgreSQL Constraints

PRIMARY KEY
Ensures each row is uniquely identified.

FOREIGN KEY
Ensures referential integrity between tables.

UNIQUE
Ensures all values in a column (or set) are unique.

NOT NULL
Ensures a column cannot contain NULL values.

CHECK
Ensures values satisfy a specific condition.

Example Table with Constraints

```
CREATE TABLE employee (
  id SERIAL PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  department_id INT NOT NULL,
  salary NUMERIC(10, 2) CHECK (salary > 0),
   CONSTRAINT fk_department
    FOREIGN KEY (department_id)
    REFERENCES department(id)
);
```

Example Constraint Violations

```
INSERT INTO employee (id, email, department_id, salary)
VALUES (1, 'john@example.com', 10, 50000);
ERROR: duplicate key value violates unique constraint
"employee_pkey"
```

```
INSERT INTO employee (id, email, department_id, salary)
VALUES (2, 'john@example.com', 10, 50000);
ERROR: duplicate key value violates unique constraint
"employee_email_key"
```

```
INSERT INTO employee (id, email, department_id, salary)
VALUES (3, 'jane@example.com', 99, 60000);
ERROR: insert or update on table "employee" violates
foreign key constraint "fk_department"
```

```
INSERT INTO employee (id, email, department_id, salary)
VALUES (4, NULL, 10, 60000);
ERROR: null value in column "email" violates not-null
constraint
```

```
INSERT INTO employee (id, email, department_id, salary)
VALUES (5, 'sam@example.com', 10, -100);
ERROR: new row for relation "employee" violates check
constraint "employee_salary_check"
```

Why Constraints Matter

- Maintain data integrity
- Prevent invalid data
- Detect issues early
- Enforce business rules

Key Takeaway
Understanding and handling PostgreSQL constraint violations helps you enforce data integrity, provide meaningful feedback, and build robust, reliable enterprise applications.

Use constraints to enforce data rules at the database level and protect your application from invalid data.

DATABASE MIGRATIONS OVERVIEW

A migration is a versioned, incremental, reproducible schema change -- the alternative to letting Hibernate guess the schema.

Versioned

Each change is a numbered file, applied in order, exactly once, on every environment.

Repeatable

Running the same migration set on a fresh database always produces the identical schema.

Team-Safe

Every developer, and every environment, converges on the same schema history through source control.

Not Hibernate's Job

ddl-auto: validate, from earlier this module, means Hibernate checks, never generates, schema.

KEY TAKEAWAY A migration tool, not Hibernate's ddl-auto, is the real source of truth for schema changes -- this is exactly why ddl-auto: validate was recommended earlier this module.

Database Migrations Overview

Database migrations are version-controlled changes to your database schema. They ensure consistent, repeatable, and reliable database changes across all environments.

Why Database Migrations?

- Version control for schema changes
- Consistent database state across environments
- Automated and repeatable deployments
- Safe collaboration across development teams
- Easier rollbacks and audit of changes

How Database Migrations Work

1. **Create Migration** — Define schema changes in a migration file (SQL).
2. **Version Control** — Store migration files in version control (e.g., Git).
3. **Apply Migration** — Tool executes migrations in order on the database.
4. **Track Changes** — Tool records applied migrations in a tracking table (e.g., schema_history).
5. **Consistent State** — Database schema is up-to-date and in sync across all environments.

Popular Migration Tools

Flyway

- Simple, lightweight, and easy to use
- SQL-based migrations
- Excellent for small to large applications
- Works with many databases, including PostgreSQL

Liquibase

- Supports SQL, YAML, XML, JSON formats
- Advanced features (preconditions, rollback, etc.)
- Strong community and enterprise support
- Database-agnostic and extensible

Migration Best Practices

- Keep migrations small, atomic, and focused.
- Use meaningful versioned filenames (e.g., `V1__create_employee_table.sql`).
- Test migrations in lower environments first.
- Never modify applied migrations; create new ones.
- Always support rollbacks when possible.

Migration Tracking Table (Example)

installed_rank	version	description	installed_on	success
1	V1	create_employee_table	2025-05-01 10:15:30	true
2	V2	add_department_table	2025-05-01 10:15:31	true
3	V3	add_salary_column	2025-05-01 10:15:33	true
...	...	...	...	...

Rollbacks

- Rollback to a previous state when a migration fails.
- Flyway: use undo scripts or repair when necessary.
- Liquibase: built-in rollback support.
- Always test rollbacks in non-production first.

Key Takeaway

Database migrations bring structure, consistency, and confidence to schema changes. They are essential for modern DevOps and continuous delivery practices.

FLYWAY/LIQUIBASE CONCEPT

Flyway, the migration tool this course uses, applies versioned SQL files in strict numeric order and tracks which have already run.

```
-- db/migration/V1__crm_schema.sql
CREATE TABLE customer ( ... );   -- from the Module 37 design
CREATE TABLE account ( ... );    -- from the Module 37 design

-- db/migration/V2__add_account_status.sql
ALTER TABLE account ADD COLUMN status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE';
```

KEY TAKEAWAY Flyway's flyway_schema_history table is the record of exactly which migrations have already run -- a fresh environment applies every file from V1 forward; an existing one applies only what's new.

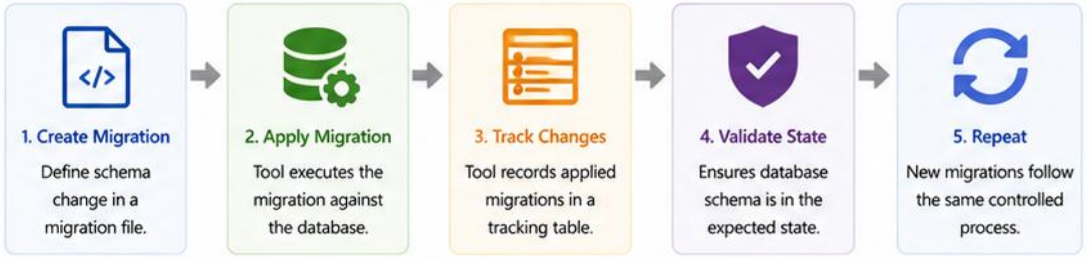
Flyway / Liquibase Concept

Flyway and Liquibase are database migration tools that help manage schema changes in a version-controlled, repeatable, and automated way.

Key Benefits

- Version control for database schema changes
- Repeatable and reliable deployments
- Works across all environments (Dev, Test, Prod)
- Auditable history of schema changes
- Easy rollback (Liquibase) and repair (Flyway)
- Supports team collaboration and DevOps automation

How Database Migrations Work



How They Track Migrations

Flyway

Table: `flyway_schema_history`

- `version`
- `description`
- `type`
- `installed_on`
- `success`

Liquibase

Table: `databasechangelog`

- `id`
- `author`
- `filename`
- `dateexecuted`
- `exectype`

Flyway vs Liquibase

Feature	Flyway	Liquibase
Approach	Migration SQL scripts	Change sets (XML, YAML, JSON, SQL)
File Naming	Versioned files (`V1__init.sql`)	No strict naming (changeSets with `id`)
Configuration	Simple and convention-based	Flexible and highly configurable
Rollback	Limited (manual or repair)	Built-in rollback support
Supported DBs	Many (including PostgreSQL)	Very many (including PostgreSQL)
Best For	Simple projects, SQL-first teams	Complex projects, enterprise needs

Best Practices

- Keep migrations small and focused.
- Always test migrations in lower environments first.
- Never modify applied migrations; add new ones.
- Review and commit migration files to version control.

Key Takeaway

Flyway and Liquibase bring structure and automation to database schema management.
Use them to ensure consistent, reliable, and auditable database changes across your applications.

SCHEMA MANAGEMENT STRATEGY

A consistent policy across environments -- validate in every environment, never auto-generate anywhere real data matters.

Environment	ddl-auto	Schema Source
Local development	validate	Flyway migrations, same as every other environment.
CI / automated tests	validate	Flyway migrations applied fresh against a test database.
Staging / Production	validate	Flyway migrations, applied as a deliberate, reviewed deployment step.

KEY TAKEAWAY The same policy, validate everywhere plus Flyway everywhere, applies uniformly across every environment -- consistency across environments is the entire point, not an environment-specific relaxation.

Schema Management Strategy

A well-defined schema management strategy ensures database schema changes are version-controlled, consistent across environments, and safe for production deployments.

Strategy Principles

- **Version Control** — Every schema change is versioned and tracked.
- **Consistency** — Same schema across Dev, Test, Staging, and Prod.
- **Automation** — Automate migrations as part of CI/CD pipeline.
- **Repeatability** — Migrations can be applied repeatedly and safely.
- **Auditability** — Full history of schema changes and authors.

Schema Management Approach

1. **Design** — Model and review schema changes.
2. **Develop** — Create migration scripts (Flyway/ Liquibase).
3. **Test** — Test migrations in isolated environments.
4. **Deploy** — Apply migrations to higher environments.
5. **Monitor** — Validate schema and monitor application.

Environments and Flow

Local Dev (Developer machine) → Dev (Development Environment) → Test (Testing Environment) → Stage / UAT (Staging / UAT Environment) → Prod (Production Environment)

Migrations flow forward (no manual schema changes)

Recommended Tools

Flyway
- SQL-based migrations
- Simple and lightweight
- Convention over configuration
- Great for database-first teams

Liquibase
- Change sets (XML/YAML/JSON/SQL)
- Advanced preconditions
- Built-in rollback support
- Ideal for complex enterprise needs

Key Practices

- Never change schema manually in any environment.
- Always write migration scripts for every change.
- Test migrations in lower environments before promoting.
- Keep migrations small, atomic, and focused.
- Review and approve migrations via pull requests.
- Backup database before major schema changes.
- Monitor migration execution and application health.

Migration Naming Best Practice

```
Flyway Example:
V1_0_1__create_employee_table.sql
Liquibase Example:
20250501-1-create-employee-table.xml
```

Common Schema Change Types

Create / Alter Tables · Add / Modify Constraints · Indexes · Sequences · Views / Functions / Procedures

Key Takeaway

A strong schema management strategy with versioned migrations ensures database integrity, reduces risk, and supports reliable delivery across all environments.

Automate schema changes. Version everything. Never change schema manually in production.

TRY IT YOURSELF — EXERCISE 5: PLAN TRANSACTIONS AND MIGRATIONS

Reinforces: transaction integration, @Transactional with JPA, persistence exceptions, constraint violations, and the migration slides you just covered.

STEPS (notes/lab39-transactions-migrations.md)

Step	What To Do
1 — Boundary	Say which layer carries @Transactional and why not the repository.
2 — Constraint Violation	Say what a duplicate email produces, and what the API returns.
3 — Migration Naming	Write the Flyway file name for the initial CRM schema.
4 — Never Edit	Say what happens if an already-applied migration is edited.

Transactions and migrations

```
@Transactional on the service -- one business operation, one transaction
duplicate email -> DataIntegrityViolationException -> 409 Conflict
V1__crm_schema.sql applied; edit it later -> checksum mismatch, startup fails
```

TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-transactions-and-migrations.md (workspace: examples/module-39-exercises).

TESTING REPOSITORIES

A repository test that mocks the database proves nothing about the actual SQL Hibernate generates -- test against a real PostgreSQL instance instead.

```
@DataJpaTest
class CustomerRepositoryTest {
    @Autowired CustomerRepository repo;

    @Test
    void findByStatus_returnsOnlyMatchingCustomers() {
        repo.save(activeCustomer("Amina Khan"));
        repo.save(prospectCustomer("Ravi Singh"));
        assertThat(repo.findByStatus("ACTIVE")).hasSize(1);
    }
}
```

KEY TAKEAWAY @DataJpaTest loads just the JPA layer, not the whole application context, and runs genuinely against a real database -- proving the derived query, the entity mapping, and the schema all actually agree.

Testing Repositories

Testing repositories ensures that data access logic works correctly, queries return expected results, and database interactions are reliable.

Key Benefits

- Verify CRUD operations and custom queries
- Ensure query methods return correct results
- Validate relationships and constraints
- Catch issues early with fast feedback
- Improve confidence in data layer stability

Types of Repository Tests

CRUD Tests
Test saved, find, update, and delete operations.

Query Method Tests
Test derived query methods and custom queries.

Relationship Tests
Test entity relationships and fetch behavior.

Constraint Tests
Test unique constraints, not-null, and validations.

Example Repository

```
public interface EmployeeRepository
  extends JpaRepository<Employee, Long> {

  List<Employee> findByDepartment(String department);
  List<Employee> findBySalaryGreaterThan(Double salary);
  Optional<Employee> findByEmail(String email);
  boolean existsByEmail(String email);
}
```

Common Annotations Used

@DataJpaTest
Configures JPA components and an in-memory database for tests.

@AutoConfigureTestDatabase
Controls test database replacement (e.g., H2, Testcontainers).

@Transactional
Rolls back transactions after each test to keep tests isolated.

Testcontainers
Use real PostgreSQL instance for integration-style repository tests.

Example Repository Test (JUnit 5)

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.ANY)
class EmployeeRepositoryTest {

   @Autowired
   private EmployeeRepository employeeRepository;

   @Test
   void testFindByDepartment() {
      List<Employee> employees = employeeRepository
          .findByDepartment("Engineering");
      assertFalse(employees.isEmpty());
      assertEquals("Engineering", employees.get(0).getDepartment());
   }
}
```

Repository Testing Best Practices

- Use @DataJpaTest for fast and focused repository tests.
- Keep tests isolated and transactional.
- Use meaningful test data and clear test names.
- Prefer Testcontainers for production-like testing.
- Avoid testing implementation details.
- Focus on behavior, not how the query is implemented.

Key Takeaway
Well-tested repositories provide confidence in data access logic, reduce production defects, and support maintainable code.

Test early, test often, and trust your data layer.

POSTGRESQL TEST DATABASE

Testing against a real PostgreSQL instance, not an in-memory substitute, is what actually proves the schema and entities agree.

An in-memory database (like H2) speaks a different SQL dialect -- a test can pass there and fail against real PostgreSQL, or the reverse.
PostgreSQL-specific features from Module 37 -- JSONB, TIMESTAMPTZ, sequences, real constraint error codes -- often have no in-memory equivalent at all.
A dedicated, disposable test database (a local Docker container, or a shared CI instance) keeps tests realistic without risking real data.
Flyway applies the same migration files to the test database as every other environment, proving the migration history itself is correct.

KEY TAKEAWAY Testing against a genuine PostgreSQL instance is what makes a green test suite actually mean something -- an in-memory substitute can hide a real bug that only appears in production.

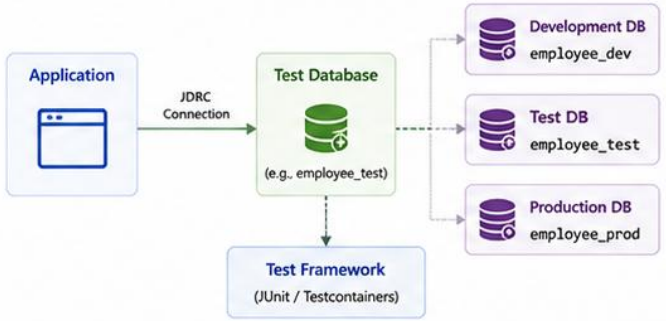
PostgreSQL Test Database

A dedicated PostgreSQL test database provides an isolated, repeatable, and reliable environment for automated testing.

Why Use a PostgreSQL Test Database?

- Isolates tests from development and production data
- Ensures consistent and repeatable test results
- Allows safe schema changes during tests
- Improves test reliability and confidence
- Supports parallel test execution
- Keeps test data clean and easy to manage

Test Database Isolation



Best Practices

- Use a separate database for tests.
- Keep test data isolated and minimal.
- Reset database state between tests.
- Use migrations to create the test schema.
- Use Testcontainers for consistent environments.
- Avoid sharing test databases across teams.
- Clean up resources after tests complete.

Example Test Database Setup

1. **Create a dedicated database**
   `CREATE DATABASE employee_test;`
2. **Create a dedicated user (optional)**
   `CREATE USER test_user WITH PASSWORD 'test123';`
3. **Grant necessary privileges**
   `GRANT ALL PRIVILEGES ON DATABASE employee_test TO test_user;`
4. **Configure application properties**
   spring.datassource.url=jdbc:postgresql://localhost:5432/employee_test
5. **Run migrations to initialize schema**
   Flyway/Liquibase applies schema to the test database.

Example application-test.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/employee_test
spring.datasource.username=test_user
spring.datasource.password=test123
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.hibernate.ddl-auto=validate   # Use migrations
spring.jpa.show-sql=true
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
spring.flyway.clean-disabled=false      # Allow clean in tests only
```

Tools for Test Databases

Testcontainers
Spin up real PostgreSQL in Docker for integration tests. Ensures consistency across all environments.

Flyway / Liquibase
Manage schema migrations for test databases.

Spring @DataJpaTest
Provides lightweight JPA testing with an embedded or containerized database.

Key Takeaway

A well-configured PostgreSQL test database ensures isolated, reliable, and maintainable tests, leading to higher quality and confidence in your application.

Test in isolation. Test with confidence. Deliver quality.

PERFORMANCE CONSIDERATIONS

Everything Module 38 taught about SQL performance still applies -- Hibernate just generates the SQL now instead of a developer typing it.

Watch the Generated SQL

spring.jpa.show-sql=true, then apply Module 38's EXPLAIN ANALYZE habits directly.

Guard Against N+1

The single most common JPA-specific performance bug -- catch it in development, not production.

Paginate Everything

Never call findAll() on a table expected to grow large.

Index What JPA Queries

Foreign keys and derived-query filter columns still need real PostgreSQL indexes, exactly as in Module 37.

KEY TAKEAWAY JPA adds a productive layer on top of SQL; it does not remove the need to think about performance -- every Module 38 habit still applies, just one layer removed.

Performance Considerations

Good performance starts with the right mappings, queries, indexes, and database usage. Optimize early and measure continuously.

Key Areas That Impact Performance

- **Database** — Indexing, query performance, statistics
- **JPA/Hibernate** — Fetch strategies, N+1 problem, caching
- **Application** — Efficient code, pagination, batching
- **Monitoring** — Measure, analyze, and tune continuously
- **Infrastructure** — Connection pooling, hardware, configuration

Best Practices

- Design proper indexes for frequent queries.
- Use appropriate fetch strategies (LAZY/EAGER).
- Avoid N+1 select problem.
- Use pagination for large result sets.
- Enable query caching for read-heavy data.
- Use batch operations for inserts/updates.
- Analyze and optimize slow queries.
- Monitor application and database metrics.

JPA / Hibernate Considerations

- **Fetch Strategy** — Default LAZY for @ManyToOne and @OneToMany. Fetch only what you need.
- **N+1 Problem** — Use JOIN FETCH or @EntityGraph to avoid multiple queries.
- **Caching** — Enable 2nd level cache and query cache where appropriate.
- **Batching** — Use batch_size and JDBC batching for bulk operations.

Database Considerations (PostgreSQL)

- **Indexes** — Create indexes on WHERE, JOIN, and ORDER BY columns. Avoid over-indexing.
- **Query Optimization** — Write efficient SQL. Avoid functions on indexed columns. Use EXPLAIN ANALYZE to analyze queries.
- **Statistics** — Keep statistics up to date using ANALYZE. Helps the query planner choose the best plan.
- **Configuration** — Tune shared_buffers, work_mem, effective_cache_size, and other parameters based on workload.

Common Performance Issues

- N+1 select problem
- Fetching unnecessary data
- Missing or poorly designed indexes
- Large result sets without pagination
- Overusing EAGER fetching
- Inefficient queries (e.g., SELECT *)

Measuring and Monitoring

- **Application** — Use Spring Boot Actuator, Micrometer, and custom timers.
- **Database** — Use pg_stat_statements, EXPLAIN ANALYZE, and logs.
- **Tools** — PgAdmin, DBeaver, New Relic, Prometheus, Grafana.

Quick Tips

- Start with LAZY loading.
- Use DTO projections for read-only views.
- Page large datasets (Pageable).
- Test performance early and regularly.
- Monitor in production and tune iteratively.

Key Takeaway

Performance is a combination of smart design, efficient queries, proper indexing, and continuous monitoring. Optimize based on data, not guesswork.

Measure. Analyze. Optimize. Repeat.

JPA VS NATIVE SQL TRADE-OFFS

Neither approach wins universally -- choose deliberately based on the specific query's needs, not habit.

Consideration	Favors JPA/JPQL	Favors Native SQL
Simple CRUD and lookups	Yes -- derived queries, minimal code.	Overkill.
Portability across databases	Yes -- entity-oriented, dialect-independent.	No -- tied to PostgreSQL syntax.
PostgreSQL-specific features	No -- JPQL has no equivalent.	Yes -- JSONB operators, window functions, CTEs.
Hand-tuned, EXPLAIN-verified performance	Limited control over generated SQL shape.	Yes -- full control, exactly the SQL that was measured.

KEY TAKEAWAY The right default for this course is JPA/JPQL for routine work, with a deliberate, documented switch to native SQL only when a specific row in this table's right column actually applies.

JPA vs Native SQL Trade-Offs

Choose the right data access approach based on your application requirements.

Criteria	JPA (Spring Data JPA)	Native SQL
Abstraction Level	High-level abstraction over the database. Object-oriented and database agnostic.	Low-level access. Tightly coupled to the database.
Productivity	High productivity. Less boilerplate with repositories and derived queries.	More code and manual mapping required. Slower development for complex CRUD.
Type Safety	Strong type safety with entities and repositories. Compile-time validation.	Weak type safety. Query errors are caught at runtime.
Query Capabilities	Good for standard CRUD and simple queries. Limited for complex, DB-specific features.	Full power of SQL. Ideal for complex queries, aggregates, CTEs, window functions, JSON operations.
Performance	Good for most use cases. Overhead due to ORM, dirty checking, and entity management.	Best performance for read-heavy and complex queries. No ORM overhead.
Portability	Database agnostic (works across vendors) with minimal changes.	Database specific. Queries need changes when switching vendors.
Maintainability	High maintainability. Cleaner code and consistent patterns.	Lower maintainability for large query sets. Harder to refactor and test.
Best Use Cases	Most CRUD operations, simple queries, business logic with entity relationships.	Reporting, analytics, complex joins, bulk operations, DB-specific features.

Key Takeaway: Use JPA for developer productivity, maintainability, and standard operations.
Use Native SQL when you need maximum performance or advanced database capabilities.

ENTERPRISE PERSISTENCE BEST PRACTICES

A consolidated checklist -- every practice here traces back to a specific slide earlier this module.

Practice	In Short
Let Flyway Own the Schema	ddl-auto: validate everywhere; every change is a reviewed migration file.
Default Associations to LAZY	Override JPA's EAGER defaults deliberately; use JOIN FETCH when eager loading is genuinely needed.
Guard Against N+1	Watch generated SQL; fetch related data intentionally, not accidentally in a loop.
Keep Transactions at the Service Layer	@Transactional on services, never repositories; understand the rollback rules.
Bound Every Paged Query	Cap page size; always add a unique sort tie-breaker.
Test Against Real PostgreSQL	No in-memory substitutes -- prove the schema and entities actually agree.

KEY TAKEAWAY Every row in this table is a direct callback to a slide already covered -- treat this as the pre-submission checklist for Lab 39.

ENTERPRISE PERSISTENCE BEST PRACTICES

Follow proven patterns to build robust, scalable, secure, and maintainable data access layers.

1. Model for the Business
Design entities that reflect business domain and rules clearly.

2. Use Appropriate Relationships
Choose correct cardinality and cascade types. Avoid overly complex object graphs.

3. Fetch Strategically
Use LAXY loading by default. Avoid N+1 queries with JOIN FETCH or Entity Graphs.

4. Index for Performance
Index frequently queried columns, foreign keys, and unique constraints. Review and tune regularly.

5. Write Efficient Queries
Use derived queries for simple cases. Use JPQL/Native SQL for complex or performance-critical queries.

6. Ensure Data Integrity
Use constraints (PK, FK, UNIQUE, NOT NULL, CHECK) to protect data quality.

7. Manage Transactions
Define clear transaction boundaries. Use @Transactional appropriately. Keep transactions short.

8. Handle Exceptions Gracefully
Translate persistence exceptions. Provide meaningful error messages to callers.

9. Monitor and Profile
Monitor slow queries and DB metrics. Use EXPLAIN ANALYZE and logging to identify bottlenecks.

10. Secure Data Access
Enforce least privilege on DB users. Protect sensitive data. Use parameterized queries to prevent SQL injection.

Additional Guidelines
Keep entities small and cohesive. Avoid anemic domain models.

Plan for Growth
Design schemas and queries that scale with data volume.

Test and Validate
Test repositories and queries with real data scenarios.

Maintain and Evolve
Review mappings, queries, and indexes regularly as requirements evolve.

TRY IT YOURSELF — EXERCISE 6: PLAN REPOSITORY TESTS AND SELF-CHECK

Pre-lab gate: plan the PostgreSQL repository tests Lab 39 grades, then confirm the five earlier notes files are genuinely complete.

STEPS (notes/lab39-readiness.md)

Step	What To Do
1 — Test Cases	Name three CustomerRepositoryIT cases, including one that must fail.
2 — Real PostgreSQL	Say why these tests run against PostgreSQL and not H2.
3 — Open-in-View	Say what open-in-view is and why this cohort turns it off.
4 — Pass Mark	Mark Pass only if all five earlier notes files are complete, not stubbed.

Repository test plan

```
findByEmail returns Amina CUS-1001
duplicate email insert -> DataIntegrityViolationException
open-in-view=false: lazy loads fail in tests, not silently in the view
```

TRY IT NOW — Complete Exercise 6 before Lab 39: exercises/exercise-06-lab39-readiness.md (workspace: examples/module-39-exercises).

LAB OVERVIEW — SPRING DATA JPA AND POSTGRESQL INTEGRATION

Wire Spring Data JPA to PostgreSQL with entities, repos, paging, migrations |

BUSINESS SCENARIO

- Leadership freezes:

LEARNING OBJECTIVES

- Add the PostgreSQL driver and configure `application.yml` for JDBC Map entities and `@OneToMany` / `@ManyToOne` with intentional fetch types Use `JpaRepository` and derived query methods Page results with `Pageable` and keep `open-in-view: false` Own schema with Flyway (`validate`) and test against PostgreSQL

WHAT YOU BUILD

- `examples/lab39-crm/` — Boot app, Flyway, IT | Flyway V1 · entities · paging API · Postgres IT green |

KEY TAKEAWAY Connect the CRM API to **PostgreSQL with **Spring Data JPA**: driver dependency, datasource config, entities, repositories, derived queries, relationships and fetch strategies, `Pageable`, Flyway migrations (Liquibase as conceptual alternative), and a real…**

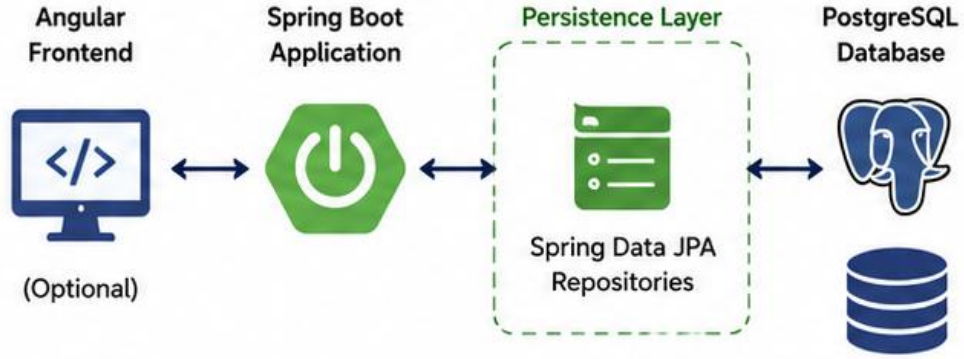
LAB OVERVIEW – SPRING DATA JPA AND POSTGRESQL

In this lab, you will build a complete data access layer using Spring Data JPA with a PostgreSQL database.

LAB OBJECTIVES

- Configure Spring Boot to connect to PostgreSQL
- Create JPA entities and relationships
- Implement Spring Data JPA repositories
- Perform CRUD operations
- Write custom queries using JPQL and native SQL
- Implement pagination and sorting
- Test repositories with integration tests

LAB ARCHITECTURE



Technologies Used

Spring Boot • Spring Data JPA • Hibernate • PostgreSQL
Maven • JUnit 5 • Testcontainers • pgAdmin (Optional)

LAB TASKS

1. Set up PostgreSQL database and connection
2. Create entities (e.g., Department, Employee)
3. Define relationships and constraints
4. Create Spring Data JPA repositories
5. Implement CRUD operations
6. Write custom queries (JPQL and native SQL)
7. Implement pagination and sorting
8. Write integration tests for repositories

PREREQUISITES

- Java 17+
- Spring Boot Fundamentals
- PostgreSQL installed (or Docker)
- IDE (IntelliJ IDEA or VS Code)
- Basic SQL knowledge

TOOLS & ENVIRONMENT

- IntelliJ IDEA / VS Code
- Spring Initializr
- PostgreSQL 14+
- pgAdmin (Optional)
- Maven
- JUnit 5 & Testcontainers

JUnit 5

PostgreSQL

EXPECTED OUTCOMES

- Working Spring Boot application connected to PostgreSQL
- JPA entities with relationships and constraints
- Repository layer with derived and custom queries
- CRUD operations verified via tests or Postman
- Pagination and sorting implemented and tested
- All repository tests passing

NOTE: This lab focuses on the persistence layer. Service and controller layers are covered in later modules.

LAB TASKS AND DELIVERABLES

Spring Data JPA and PostgreSQL Integration — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md; copy to examples/lab39-crm.
2	.env.example → .env; docker compose up -d.
3	Complete Flyway V1 TODOs, entities, repository/service TODOs.
4	mvn -B test — IT against real PostgreSQL.
5	Evidence under notes/screenshots/lab-39/.
6	Boot app with PostgreSQL driver + datasource yml/properties
7	Flyway V1__crm_schema.sql (migration-owned schema)
8	CustomerEntity / AccountEntity + relationship mapping
9	JpaRepository + derived queries + Pageable API
10	Lazy vs eager notes; open-in-view off
11	CustomerRepositoryIT green on PostgreSQL
12	Flyway/Liquibase concept note in docs/jpa-postgres-notes.md

EXPECTED DELIVERABLES

- Boot app with PostgreSQL driver + datasource yml/properties Flyway V1__crm_schema.sql (migration-owned schema) CustomerEntity / AccountEntity + relationship mapping JpaRepository + derived queries + Pageable API Lazy vs eager notes; open-in-view off CustomerRepositoryIT green on PostgreSQL Flyway/Liquibase concept note in docs/jpa-postgres-notes.md No secrets / ddl-auto=update in Git

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor Lab 39; complete pre-lab exercises 1→6 first, then the graded lab.

LAB TASKS AND DELIVERABLES

Complete the tasks below to build and validate the persistence layer using Spring Data JPA and PostgreSQL.

LAB TASKS

1. **Set Up PostgreSQL Database**
   - Create database and user.
   - Configure Spring Boot to connect to PostgreSQL.
2. **Create Entities and Relationships**
   - Create Department and Employee entities.
   - Define relationships and constraints.
3. **Create Spring Data JPA Repositories**
   - Create repository interfaces extending JpaRepository.
   - Use derived query methods.
4. **Implement CRUD Operations**
   - Implement create, read, update, and delete operations.
   - Test using Postman or a simple runner.
5. **Custom Queries**
   - Write custom queries using JPQL.
   - Write native SQL queries for advanced scenarios.
6. **Pagination and Sorting**
   - Implement pagination using Pageable.
   - Implement sorting on multiple fields.
7. **Integration Testing**
   - Write integration tests for repositories using @DataJpaTest.
   - Validate CRUD, custom queries, pagination, and sorting.

DELIVERABLES

- **Database Setup Script**
  SQL script to create database, tables, constraints, and sample data.
- **Entity Classes**
  Department.java and Employee.java with relationships and annotations.
- **Repository Interfaces**
  Spring Data JPA repository interfaces with derived query methods.
- **Service Layer (Optional)**
  Service classes implementing business logic using repositories.
- **API Test Collection**
  Postman collection to test all REST endpoints for CRUD operations.
- **Custom Query Examples**
  JPQL and native SQL query implementations with results.
- **Integration Test Suite**
  JUnit tests validating repository functionality, pagination, sorting, and custom queries.

SUCCESS CRITERIA

- Application connects successfully to PostgreSQL.
- All entities and relationships work as expected.
- CRUD operations perform correctly.
- Custom queries return correct results.
- Pagination and sorting work as expected.
- All integration tests pass successfully.

FINAL OUTCOME: A fully functional persistence layer with tested repositories, custom queries, and robust integration tests backed by PostgreSQL.

MODULE 39 SUMMARY

You can now build a tested Spring Data JPA persistence layer against real PostgreSQL.

KEY CONCEPTS COVERED

JPA Architecture

ORM, JPA, Hibernate, and Spring Data JPA -- how each layer fits together.

Entity Mapping

@Entity, @Table, @Id, @Column, and identity generation strategies.

Repositories & Queries

JpaRepository, derived queries, JPQL, and native SQL.

Relationships & Fetching

One-to-one, one-to-many, many-to-one, many-to-many, lazy vs eager.

Transactions

@Transactional, ACID, propagation, and rollback rules.

Migrations & Testing

Flyway-managed schemas and repository tests against a real database.

MODULE FLOW RECAP

1. Map
2. Query
3. Relate
4. Transact
5. Test

KEY TAKEAWAY You can now map entities to PostgreSQL tables, write repositories and queries, model relationships correctly, and wrap changes in tested, reliable transactions.

MODULE SUMMARY

This module introduced Spring Data JPA and its integration with PostgreSQL to build a robust, efficient, and maintainable persistence layer for enterprise applications.

KEY TAKEAWAYS

Simplified Data Access
Spring Data JPA reduces boilerplate code with repository abstractions and derived queries.

JPA and Hibernate
JPA provides the standard for ORM while Hibernate is the default implementation in Spring Boot.

Entity Mapping
Entities, relationships, and constraints map Java objects to PostgreSQL tables seamlessly.

Powerful Querying
Use derived query methods, JPQL, and native SQL for complex data retrieval scenarios.

Performance and Best Practices
Understand fetch strategies, indexing, pagination, transactions, and how to avoid common pitfalls.

Enterprise Ready
Test repositories, handle exceptions, manage schema migrations, and follow best practices for scalability, security, and maintainability.

WHAT WE COVERED

- JPA, Hibernate, and Spring Data JPA Overview
- PostgreSQL Integration and Configuration
- Entity Classes and Annotations
- Repository Interfaces and CRUD Operations
- Derived Queries and Custom Queries (JPQL & Native)
- Relationships and Fetch Strategies
- Pagination, Sorting, and Transactions
- Handling Exceptions and Constraint Violations
- Testing Repositories with PostgreSQL Test DB
- Database Migrations with Flyway/Liquibase
- Performance Considerations and Best Practices

TECHNOLOGIES USED

- Spring Boot
- PostgreSQL
- Spring Data JPA
- Java
- Hibernate
- Flyway / Liquibase

OUTCOMES

- Application connects to PostgreSQL successfully
- Entities and relationships are mapped correctly
- Repository layer supports all CRUD operations
- Custom queries return correct results
- Pagination and sorting work as expected
- All repository tests pass successfully

IN A NUTSHELL — You now have the skills to design and implement a production-ready persistence layer using Spring Data JPA with PostgreSQL, following enterprise best practices.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of JPA architecture, entity mapping, and repositories.

1. Which statement correctly describes the relationship between JPA and Hibernate?

A. They are two names for the same thing

B. JPA is a specification; Hibernate is one implementation of it

C. Hibernate is a specification; JPA is one implementation of it

D. JPA only works with Hibernate

2. Which annotation marks the field that uniquely identifies a JPA entity?

A. @Column

B. @Table

C. @Id

D. @Entity

3. A Spring Data JPA repository method named findByStatus(String status) is an example of:

A. A native SQL query

B. A derived query method

C. A JPQL query

D. A transaction boundary

4. Which interface gives a repository built-in save, findById, and delete methods with no extra code?

A. CrudRepository or JpaRepository

B. EntityManager

C. DataSource

D. @Transactional

KEY TAKEAWAY JPA is a specification and Hibernate is one implementation of it; @Id marks the primary key field; a method name like findByStatus is a derived query; extending JpaRepository provides CRUD methods automatically.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on relationships, fetch strategies, and transactions.

1. In a @OneToMany/@ManyToOne relationship between Customer and Account, which side owns the foreign key?

A. The One (Customer) side

B. The Many (Account) side, via @ManyToOne

C. Neither side owns it

D. Both sides own it equally

2. What is the default fetch type for a @ManyToOne or @OneToOne association?

A. LAZY

B. EAGER

C. NONE

D. It must always be specified explicitly

3. Which annotation wraps a service method so its database operations succeed or roll back together?

A. @Entity

B. @Repository

C. @Transactional

D. @Query

4. A Flyway migration file named V2__add_account_status.sql is best described as:

A. An entity class

B. A versioned, repeatable schema change tracked in source control

C. A JPQL query

D. A repository interface

KEY TAKEAWAY The Many side owns the foreign key in a one-to-many relationship; @ManyToOne and @OneToOne default to EAGER; @Transactional wraps a method's database operations into one atomic unit; a Flyway file like V2__... is a versioned schema migration.

KNOWLEDGE CHECK

Choose the best answer for each question.

1 **Which of the following is the primary goal of Spring Data JPA?**

- A To write native SQL queries only
- B To simplify data access layer development
- C To replace the database server
- D To manage database backups

2 **Which annotation marks a class as a JPA entity?**

- A @Table
- B @Entity
- C @Repository
- D @Component

3 **Which interface is extended by Spring Data JPA repositories for CRUD operations?**

- A CrudRepository
- B JpaRepository
- C EntityRepository
- D DataRepository

4 **Which method name will find employees by department name?**

- A findByDepartmentName()
- B findEmployeesByDept()
- C getByDepartment()
- D searchByDeptName()

5 **What is the default fetch type for @OneToMany relationship?**

- A EAGER
- B LAZY
- C IMMEDIATE
- D DEFAULT

6 **Which annotation ensures a method runs within a transaction?**

- A @Transactional
- B @Transaction
- C @Commit
- D @Rollback

7 **Which exception is thrown when a unique constraint is violated in PostgreSQL with JPA?**

- A DataNotFoundException
- B ConstraintViolationException
- C DuplicateKeyException
- D EmptyResultException

8 **Which tool is commonly used for database schema migrations?**

- A Lombok
- B Flyway / Liquibase
- C Postman
- D Mockito

QUICK RECAP

Spring Data JPA simplifies data access using repositories and derived queries.

Entities map Java classes to database tables using annotations.

JPA and Hibernate handle ORM, relationships, and transactions.

Pagination, sorting, and custom queries improve performance.

Best practices ensure scalability, reliability, and easy maintenance.

CHALLENGE QUESTION

How would you implement a custom query to find the top 5 highest salaried employees in each department?

Tip: Use JPQL or native SQL with grouping and ordering.

Think. Apply. Validate.

Use what you learned in this module to solve real-world persistence challenges.

MODULE 39 COMPLETE

"Focus on your business logic -- JPA takes care of the persistence."

Spring Data JPA and PostgreSQL Integration

You can now map entities, write repositories and queries, model relationships deliberately, and wrap changes in tested, reliable transactions.